

PROGRAMMING WITH WEBGL PART 1: BACKGROUND

Lecturer: Dr. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

Objectives

- Development of the WebGL API
- WebGL Architecture
 - WebGL as a data flow machine
- Functions
 - Types
 - Formats
- Simple program

Early History of APIs

- In 1973, International Federation for Information Processing (IFIPS) formed two committees to come up with a standard graphics API
 - Graphical Kernel System (GKS)
 - 2D but contained good workstation model
 - Core
 - Both 2D and 3D
 - **GKS** adopted as ISO and later ANSI standard (1980s)
- GKS not easily extended to 3D (GKS-3D)
 - Far behind hardware development

Silicon Graphics and GL



1987 Silicon Graphics promo

- Silicon Graphics (SGI) revolutionized the graphics workstation by implementing the pipeline in hardware (1982)
- To access the system, application programmers used a library called **GL**
- With GL, it was relatively simple to program three dimensional interactive applications

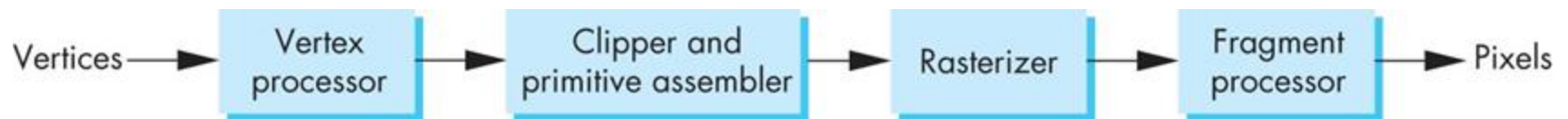
OpenGL

The success of GL lead to OpenGL in 1992,
a platform-independent API that was

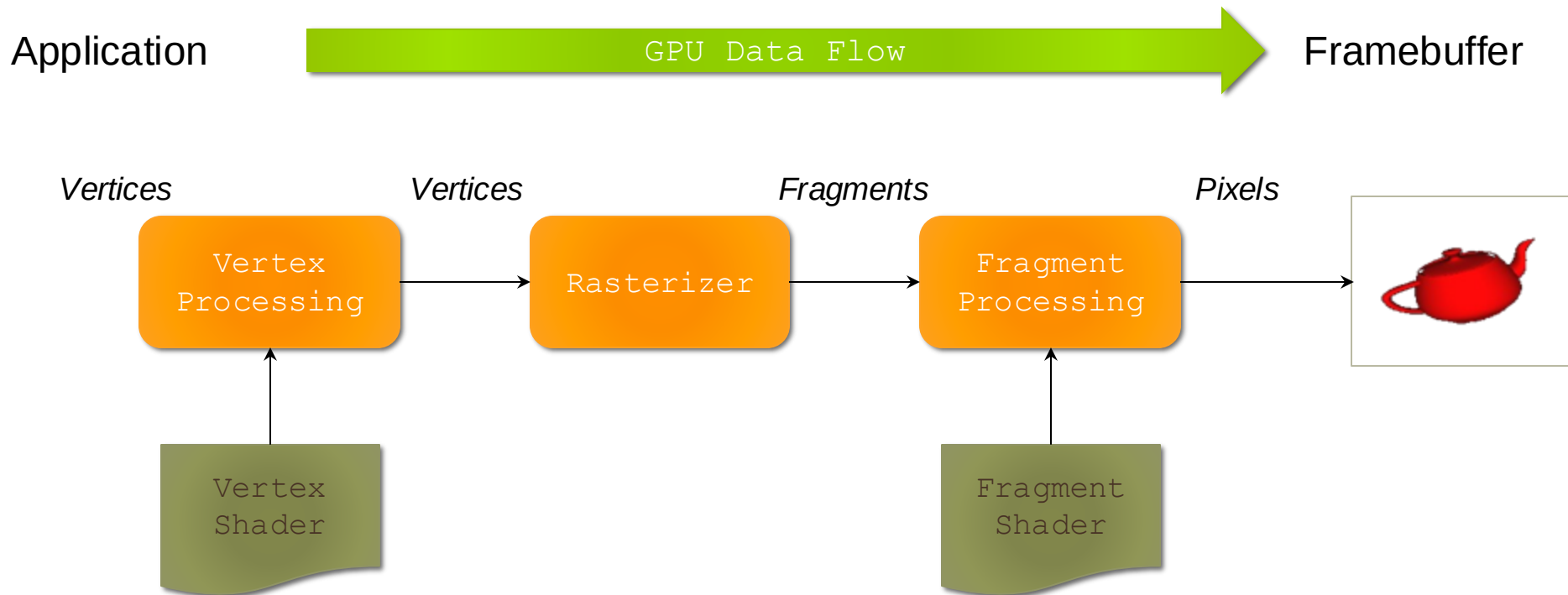
- Easy to use
- Close enough to the hardware to get excellent performance
- Focus on rendering
- Omitted windowing and input to avoid window system dependencies

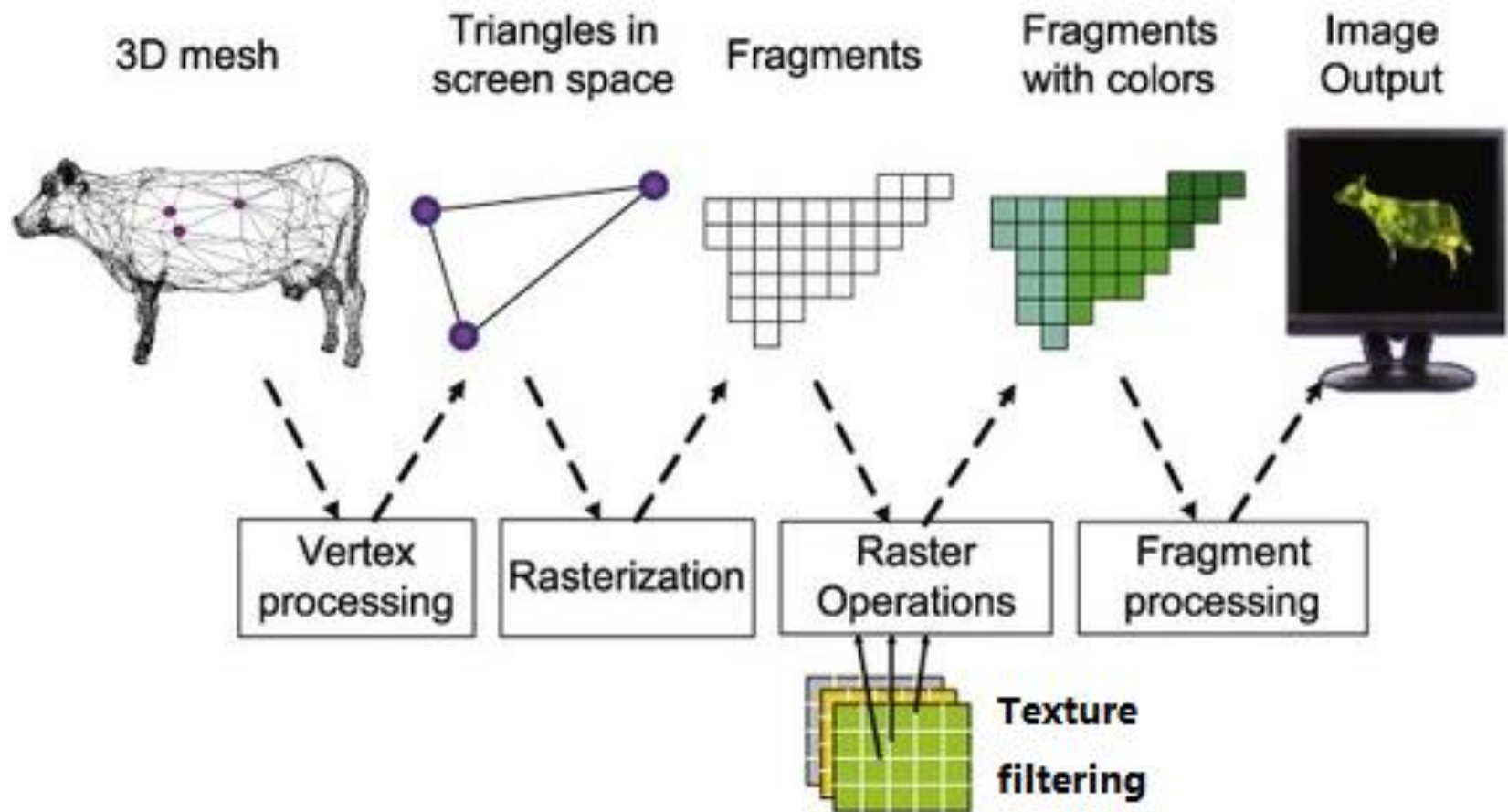
Modern OpenGL

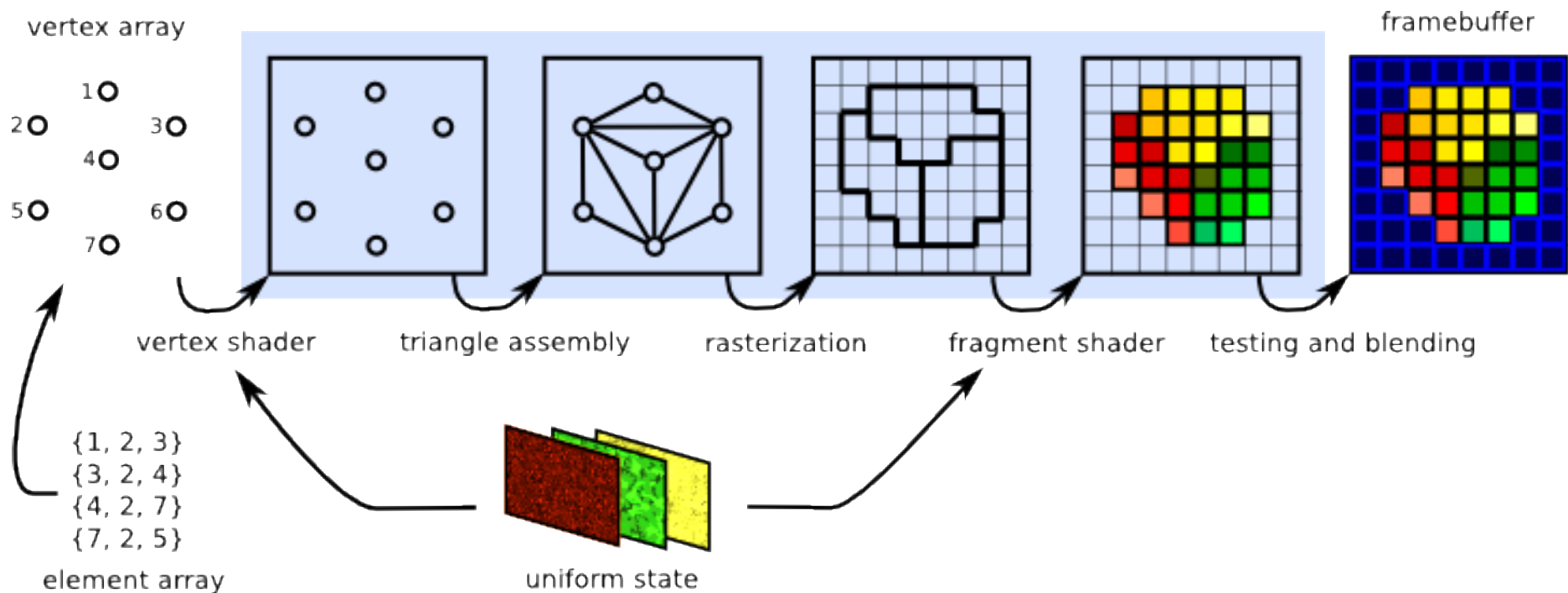
- Performance is achieved by using GPU rather than CPU
- Control GPU through programs called **shaders**
- An OpenGL application's job is: **send data to GPU**
- GPU does all rendering



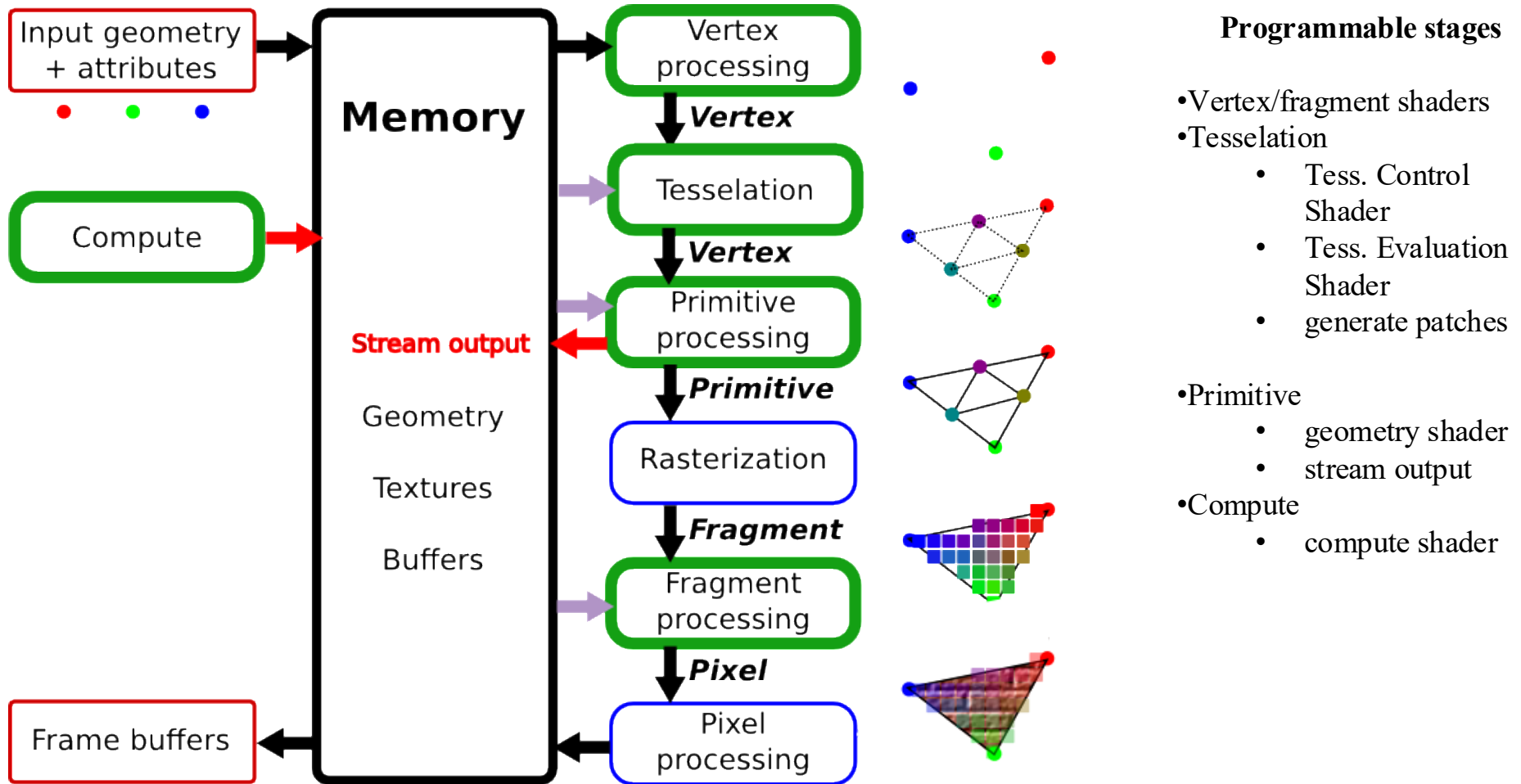
A Simplified Pipeline Model







In this example, fragments are shaded (colored) from scratch during fragment processing.



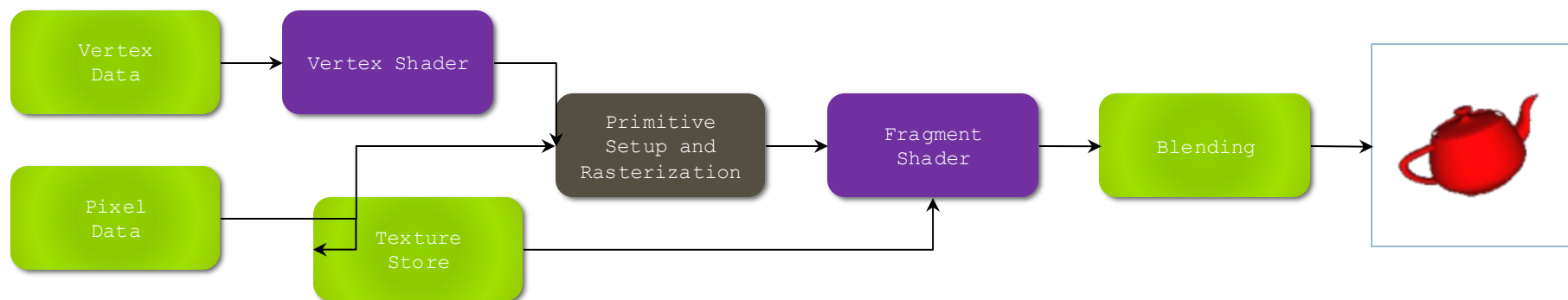
In this example, vertex colors are interpolated by the rasterizer for each fragment (potential pixel) and the interpolated colors become the temporary fragment colors. Then, the temporary colors are blended with the texture to produce the final pixel colors.

OpenGL 3.1

- Totally shader-based
 - No default shaders
 - Each application must provide both a vertex and a fragment shader
- No immediate mode
- Few state variables
- Most 2.5 functions deprecated
- Backward compatibility not required

The Exclusively Programmable Pipeline

- OpenGL 3.1 removed the fixed-function pipeline
 - Programs are required to use only shaders
 - OpenGL extensions, GL_ARB_compatibility, allows access to the removed features



- Additionally, almost all data is GPU-resident
 - all vertex data sent using buffer objects
 - reduces the impacts of various computer system architecture limitations related to GPUs

OpenGL ES

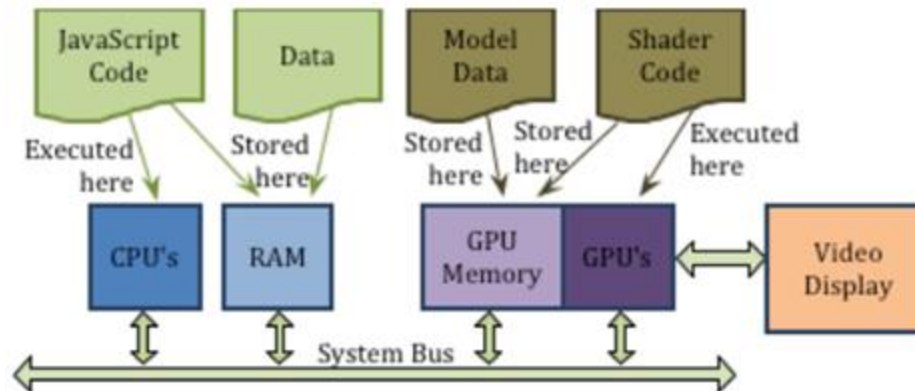
- Standard for embedded accelerated 3D graphics
- Cross platform API for rendering 2D and 3D graphics on embedded and mobile systems
 - consoles
 - phones
 - appliances
 - vehicles

WebGL

- JavaScript application programmer interface (API)
 - uses GPU of a computing device in a web browser window to render real-time 3D computer graphics
- Based on OpenGL ES 2.0
 - performs 3D rendering in an HTML <canvas> in browsers
- The only cross platform development environment for producing 3D computer graphics
- Some browsers offer better WebGL support
 - Firefox

WebGL

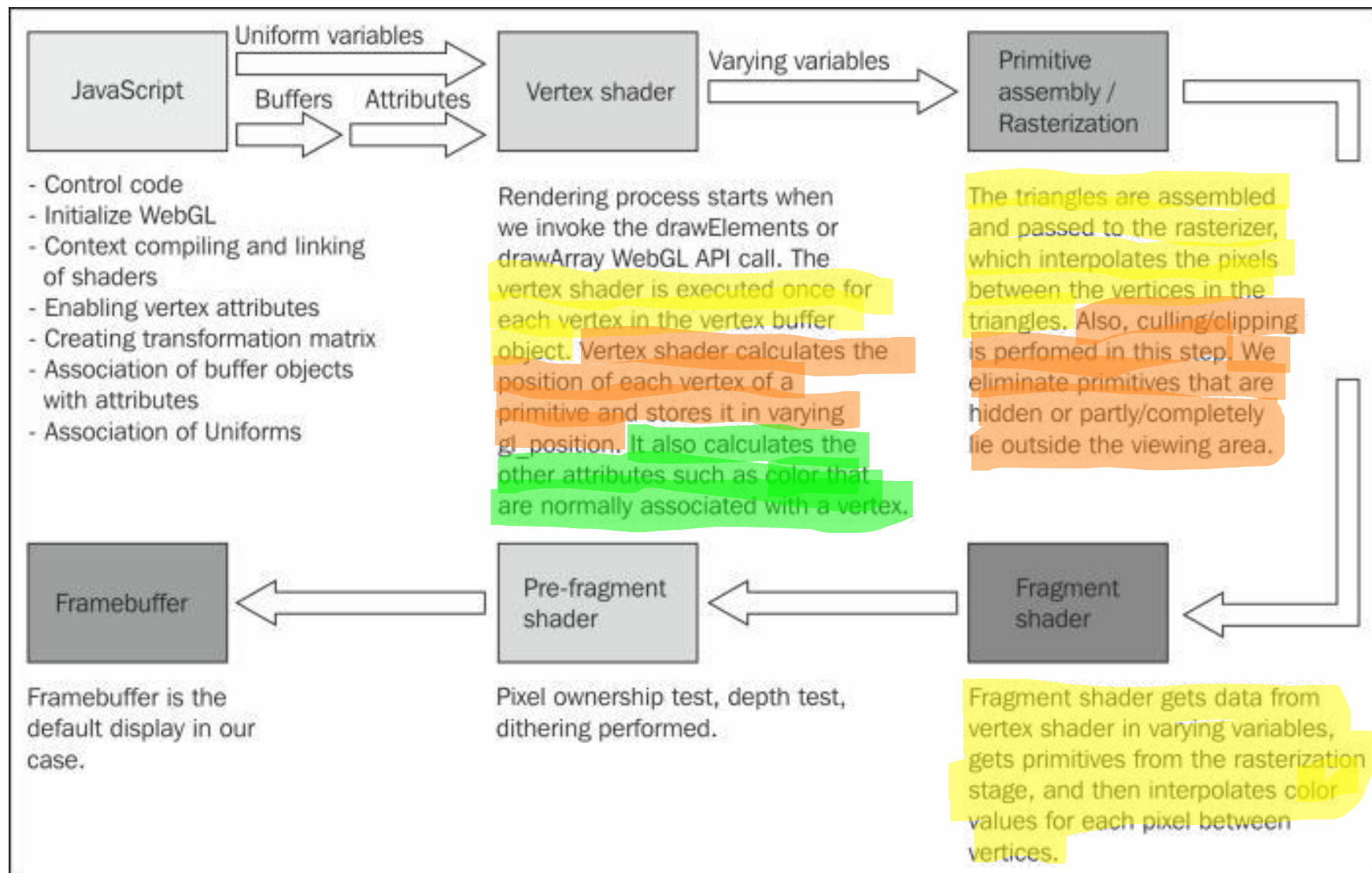
- Where the different parts of a WebGL program are stored and executed:



* Figure 5

- JavaScript code runs on the CPU(s) and the graphics rendering is done by the GPU(s)

WebGL Pipeline



What About Direct X?

- Windows only
- Advantages
 - Better control of resources
 - Access to high level functionality
- Disadvantages
 - New versions not backward compatible
 - Windows only
- Recent advances in shaders are leading to convergence with OpenGL

GLSL vs. HLSL

<https://msdn.microsoft.com/en-us/library/windows/apps/dn166865.aspx>

WebGL Environment

- **HTML**

- To run on browser that supports WebGL API
- Includes <canvas> tag to draw graphics in it

- **Javascript**

- To draw graphics into the canvas element
 - Modern browsers like chrome, firefox, safari, edge (up-to-date versions) support and run the WebGL code natively
- Using external javascript libraries is not necessary
 - But there are some like three.js, Babylon.js, etc.

- **CSS (not necessarily)**

Application Framework Requirements

- WebGL applications need a place to render into
 - A modern browser that supports it
- Some parts of API executed on CPU, the other parts executed in GPU
- A text editor to write codes and a browser to debug are enough to create a WebGL API
- We will use Netbeans IDE to create our WebGL API
 - It creates html5 / javascript project natively
 - It offers to run the project on various browsers
 - Firefox
 - Chrome
 - Edge
 - Internet Explorer

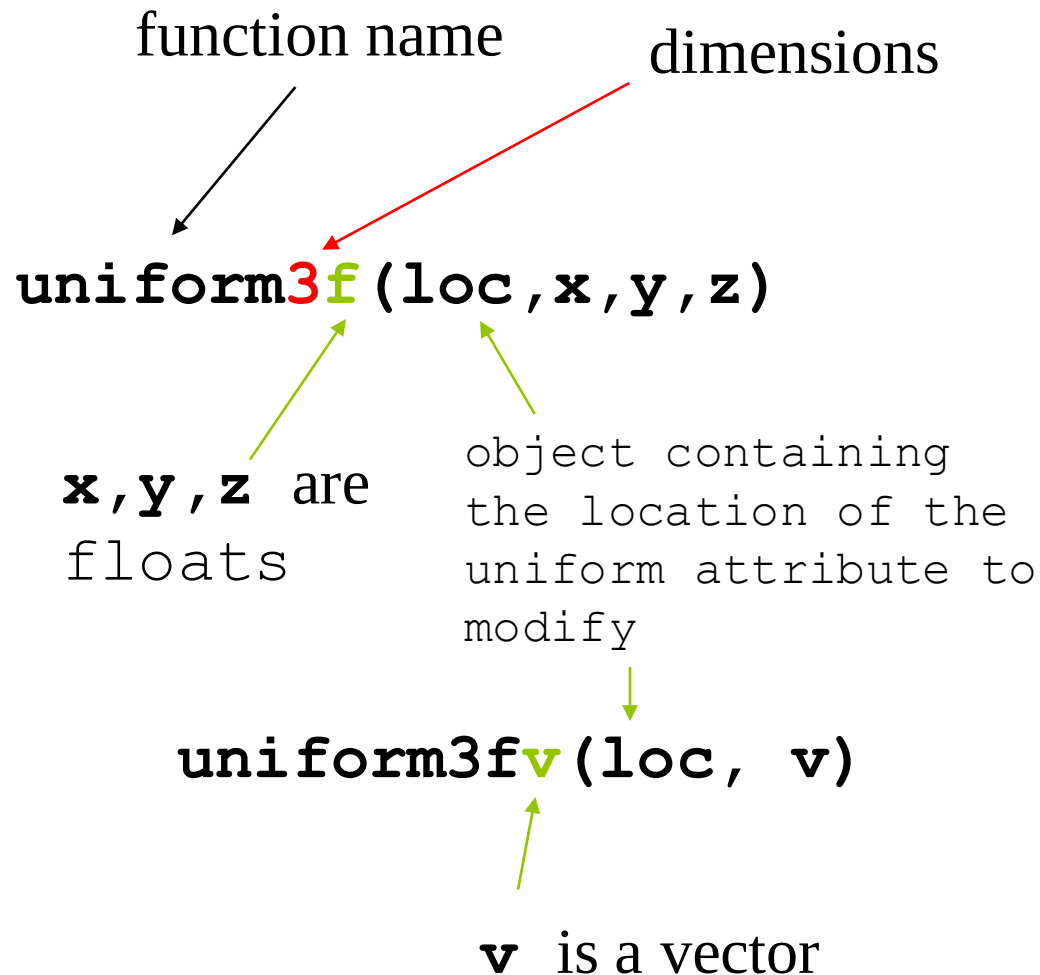
Simplifying Working with WebGL

- There are some javascript libraries including primitive data types (points, vectors, matrices) and functions (vector adding – product, matrix adding – product)
 - glMatrix from <http://glmatrix.net/>
 - three.js
 - MV.js from Angel's <https://www.cs.unm.edu/~angel/WebGL/7E/Common/>

WebGL Functions

- Primitives
 - Points
 - Line Segments
 - Triangles
- Attributes
- Transformations
 - Viewing
 - Modeling

WebGL function format



WebGL defines

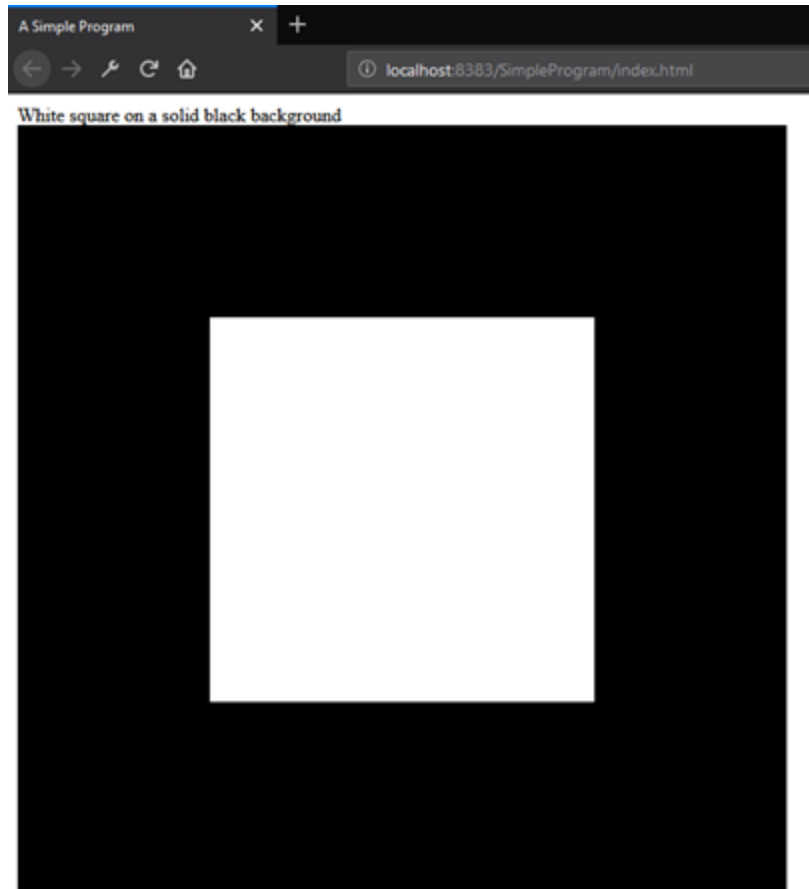
- Most constants are defined in the **WebGLRenderingContext** interface
 - Note that using a modern browser that supports WebGL should automatically include the others
 - Examples
 - `WebGLRenderingContext.enable(gl.DEPTH_TEST)`
 - `WebGLRenderingContext.clear(gl.COLOR_BUFFER_BIT)`
 - also includes WebGL data types:
 - `WebGLRenderingContext.FLOAT`,
`WebGLRenderingContext.DOUBLE`

GLSL

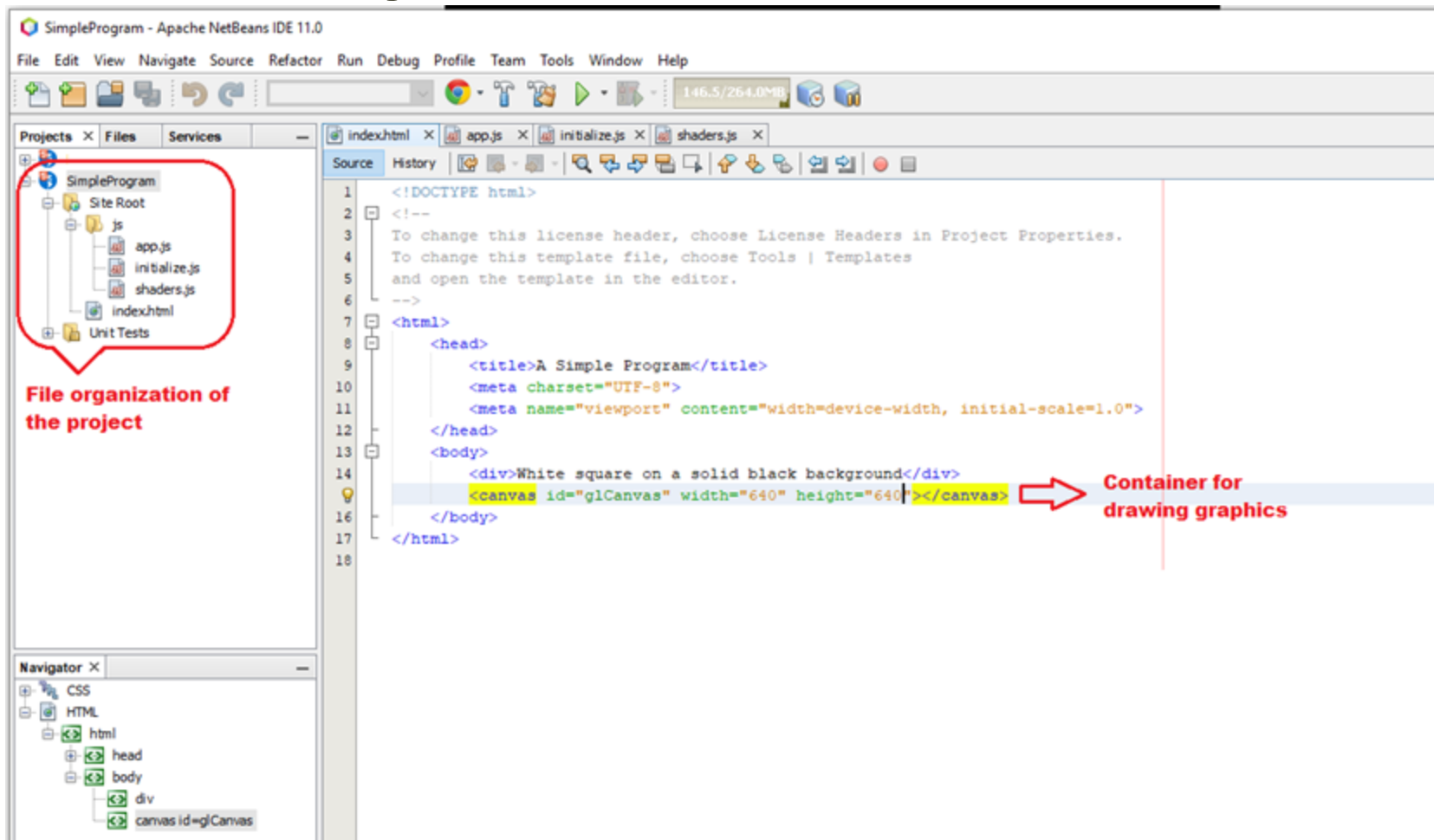
- OpenGL/WebGL Shading Language
- C-like with
 - Matrix and vector types (2, 3, 4 dimensional)
 - Overloaded operators
 - C++ like constructors
- Similar to Nvidia's Cg and Microsoft HLSL
- Code sent to shaders as source code

A Simple Program (?)

Generate a square on a solid background

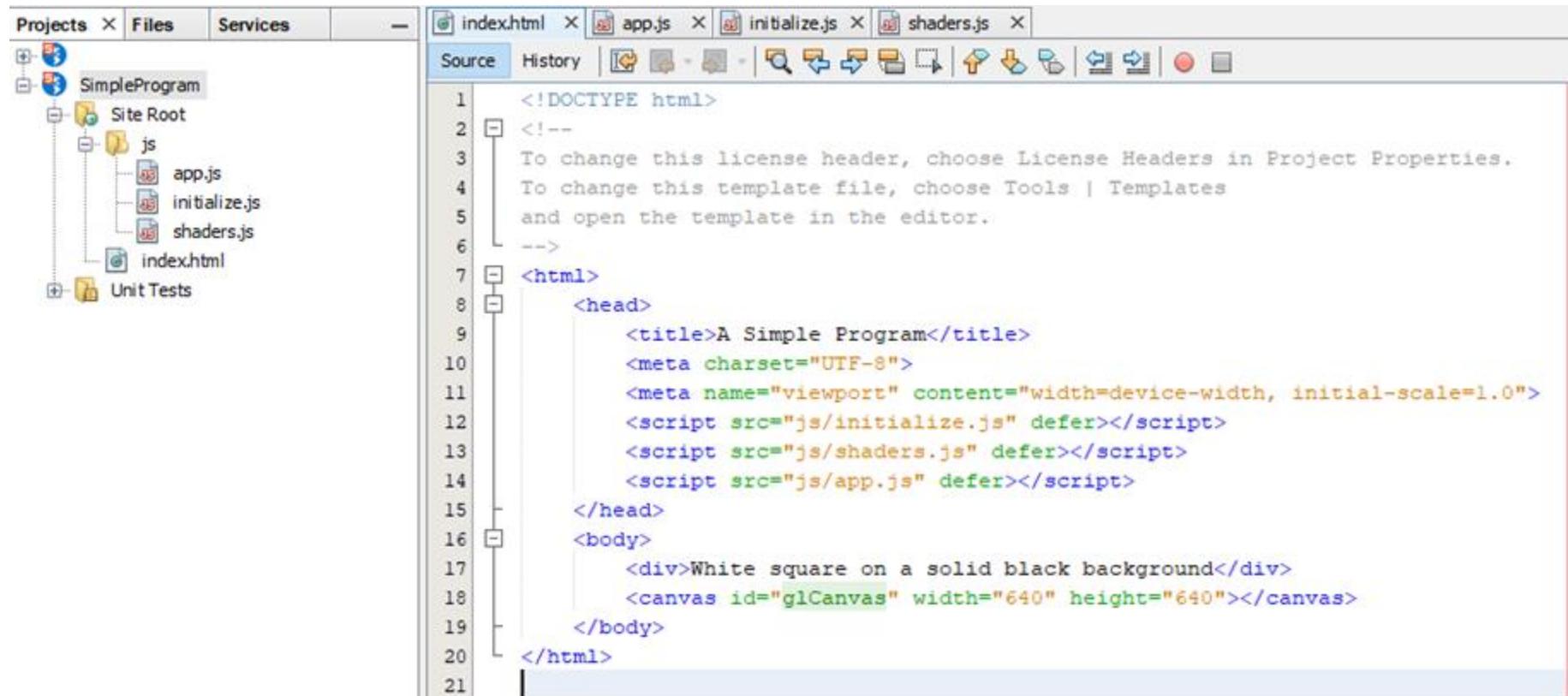


WebGL Project



- This is a html5/javascript project in Netbeans IDE
- On the left part, it shows the file organization. And html file “index.html” includes the <canvas> element as a container for drawing the graphics

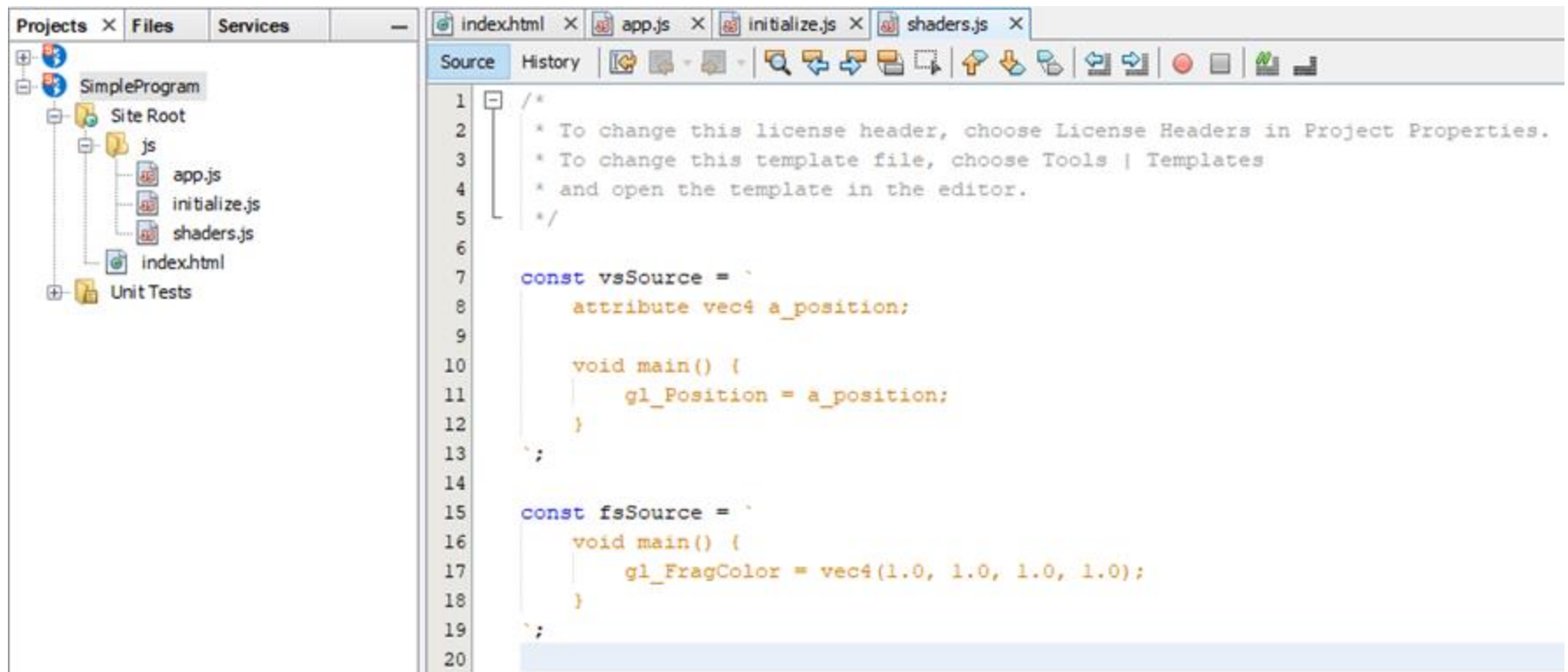
index.html



```
1 <!DOCTYPE html>
2 <!--
3 To change this license header, choose License Headers in Project Properties.
4 To change this template file, choose Tools | Templates
5 and open the template in the editor.
6 -->
7 <html>
8   <head>
9     <title>A Simple Program</title>
10    <meta charset="UTF-8">
11    <meta name="viewport" content="width=device-width, initial-scale=1.0">
12    <script src="js/initialize.js" defer></script>
13    <script src="js/shaders.js" defer></script>
14    <script src="js/app.js" defer></script>
15  </head>
16  <body>
17    <div>White square on a solid black background</div>
18    <canvas id="glCanvas" width="640" height="640"></canvas>
19  </body>
20 </html>
21
```

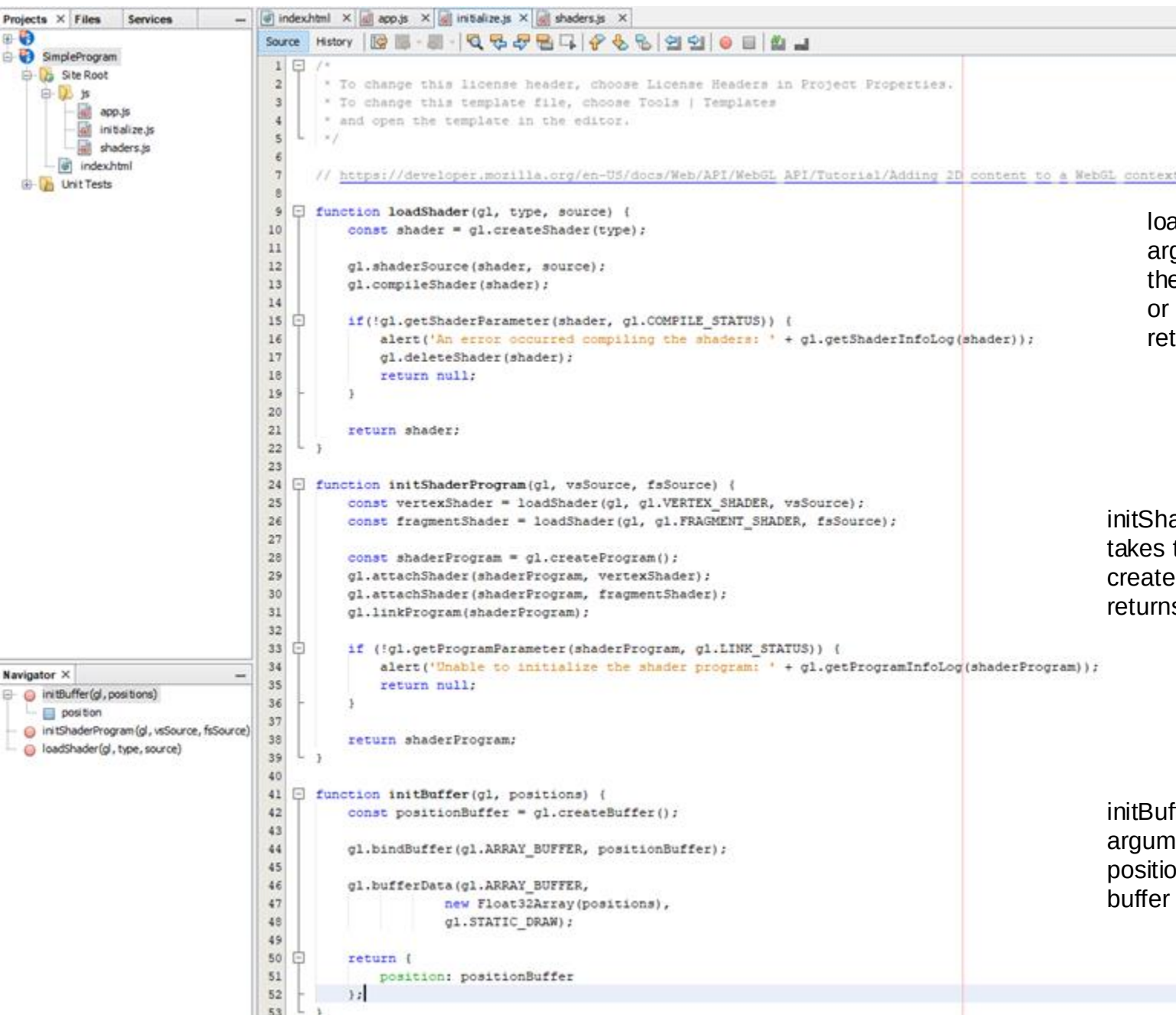
- This is the HTML code.
- We are not interested in styling the webpage but drawing graphics. So, if you want to play with the page, you had better create css folder under the “Site Root” and you can include your css files in <head> tag in html code.

shaders.js



These are the shader codes in two string constants: vertex shader and the fragment shader

Initialize.js



The screenshot shows a web browser with the following tabs: index.html, app.js, initialize.js, and shaders.js. The left sidebar shows a project structure for 'SimpleProgram' with a 'Site Root' containing 'js' (with 'app.js', 'initialize.js', and 'shaders.js') and 'Unit Tests'. The main content area displays the source code of 'initialize.js'.

```
1  /*
2   * To change this license header, choose License Headers in Project Properties.
3   * To change this template file, choose Tools | Templates
4   * and open the template in the editor.
5   */
6
7   // https://developer.mozilla.org/en-US/docs/Web/API/WebGL_API/Tutorial/Adding_2D_content_to_a_WebGL_context
8
9   function loadShader(gl, type, source) {
10     const shader = gl.createShader(type);
11
12     gl.shaderSource(shader, source);
13     gl.compileShader(shader);
14
15     if(!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
16       alert('An error occurred compiling the shaders: ' + gl.getShaderInfoLog(shader));
17       gl.deleteShader(shader);
18       return null;
19     }
20
21     return shader;
22   }
23
24   function initShaderProgram(gl, vsSource, fsSource) {
25     const vertexShader = loadShader(gl, gl.VERTEX_SHADER, vsSource);
26     const fragmentShader = loadShader(gl, gl.FRAGMENT_SHADER, fsSource);
27
28     const shaderProgram = gl.createProgram();
29     gl.attachShader(shaderProgram, vertexShader);
30     gl.attachShader(shaderProgram, fragmentShader);
31     gl.linkProgram(shaderProgram);
32
33     if (!gl.getProgramParameter(shaderProgram, gl.LINK_STATUS)) {
34       alert('Unable to initialize the shader program: ' + gl.getProgramInfoLog(shaderProgram));
35       return null;
36     }
37
38     return shaderProgram;
39   }
40
41   function initBuffer(gl, positions) {
42     const positionBuffer = gl.createBuffer();
43
44     gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
45
46     gl.bufferData(gl.ARRAY_BUFFER,
47                  new Float32Array(positions),
48                  gl.STATIC_DRAW);
49
50     return {
51       position: positionBuffer
52     };
53   }
```

The Navigator panel on the left shows the following function calls:

- initBuffer(gl, positions)
- position
- initShaderProgram(gl, vsSource, fsSource)
- loadShader(gl, type, source)

loadShader() takes three arguments to compile the shader code – vertex or fragment – and returns the shader

initShaderProgram() again takes three arguments to create shader program and returns it

initBuffer() takes two arguments for binding positions to the created buffer and returns it

app.js

```
1  "use strict";
2
3  /*
4   * To change this license header, choose License Headers in Project Properties.
5   * To change this template file, choose Tools | Templates
6   * and open the template in the editor.
7   */
8
9  main();
10
11 function main() {
12     const canvas = document.querySelector("#glCanvas");
13     const gl = canvas.getContext("webgl");
14
15     if(!gl) {
16         alert("Unable to initialize WebGL. Your browser or machine may not support it.");
17         return;
18     }
19
20     gl.clearColor(0.0,0.0,0.0,1.0);
21     gl.clear(gl.COLOR_BUFFER_BIT);
22
23     const program = initShaderProgram(gl, vsSource, fsSource);
24
25     const positions = [-0.5, 0.5, 0.5, 0.5, -0.5, -0.5, 0.5, -0.5];
26
27     const buffer = initBuffer(gl, positions);
28
29     const numComponents = 2;
30     const type = gl.FLOAT;
31     const normalize = false;
32     const stride = 0;
33     const offset = 0;
34     const vertexCount = 4;
35
36     gl.useProgram(program);
37     gl.enableVertexAttribArray(gl.getAttribLocation(program, "a_position"));
38     gl.bindBuffer(gl.ARRAY_BUFFER, buffer.position);
39     gl.vertexAttribPointer(gl.getAttribLocation(program, "a_position"), numComponents, type, normalize, stride, offset);
40     gl.drawArrays(gl.TRIANGLE_STRIP, offset, vertexCount);
41 }
```

Do not forget!

- This is our main function that draws the graphic part.
- We need to get canvas element to be able to use webgl context.
- Then, we draw the graphics after initializing the program and buffer.

main(): undefined

Notes on compilation

- No need to compile!
- Write and test
 - Open your html file with a modern browser that supports webgl

Program Structure

- Most WebGL programs have a similar structure
 - **get the context of canvas**
 - returns webgl object
 - **load shaders**
 - compiles vertex and fragment shaders
 - **link program**
 - attaches shaders to the program and links it
 - **bind buffers**
 - binds the data to the buffer
 - **draw graphics**
 - draw the elements

Simple program revisited

- *loadShader()* will compile the shader
 - can be used commonly
- *initShaderProgram()* will create program and bind the shaders to the program
 - can be used commonly
- *initBuffer()* will create the buffer and bind the positions to it
 - can be used commonly, can have more stuff too
- ***main()***
 - specific to the application
- application program's job:
 - describe the geometry: tell where the vertices are, how they are connected..
 - can also describe how that geometry should look: color, lighting, textures
- Key issue is that we **must form a data array** to send to GPU and then render it

main()

get canvas element **glCanvas**



```
function main() {
```

```
  const canvas = document.querySelector("#glCanvas");
```

```
  const gl = canvas.getContext("webgl"); ← get webgl object
```

```
  if(!gl) {
```

```
    alert("...");
```

```
    return;
```

```
  }
```

alert a warning and return if there
is any problem with webgl support

```
  gl.clearColor(0.0,0.0,0.0,1.0); ← color the background black
```

```
  gl.clear(gl.COLOR_BUFFER_BIT);
```

clear the color buffer

```
  const program =
```

```
    initShaderProgram(gl,vsSource,fsSource); ← init program
```

```
  const positions = [...]; ← set the positions
```

```
main()
```

bind buffer with the positions

```
const buffer = initBuffer(gl, positions);
```

```
const numComponents = 2;
```

```
const type = gl.FLOAT;
```

```
const normalize = false;
```

```
const stride = 0;
```

```
const offset = 0;
```

```
const vertexCount = 4;
```

- pull out 2 values per iteration
- the data in the buffer is 32bit floats
- don't normalize
- how many bytes to move from one set of values to the next
- how many bytes inside the buffer to start from

```
gl.useProgram(program);
```

bind program to the gl object

```
gl.enableVertexAttribArray(gl.getAttributeLocation(program, "a_position"));
```

enables the generic vertex attribute array specified by index

```
gl.bindBuffer(gl.ARRAY_BUFFER, buffer.position);
```

bind the position of the buffer to the gl object

`main ()` specifies the location and data format of the array of generic vertex attributes at the index to use when rendering

```
gl.vertexAttribPointer(gl.getAttributeLocation(program, "a_
position"), numComponents, type, normalize, stride,
offset);
```

```
gl.drawArrays(gl.TRIANGLE_STRIP, offset, vertexCount);
```

Initiates rendering using specific geometric primitives with a few subroutine calls

```
}
```

Initialization

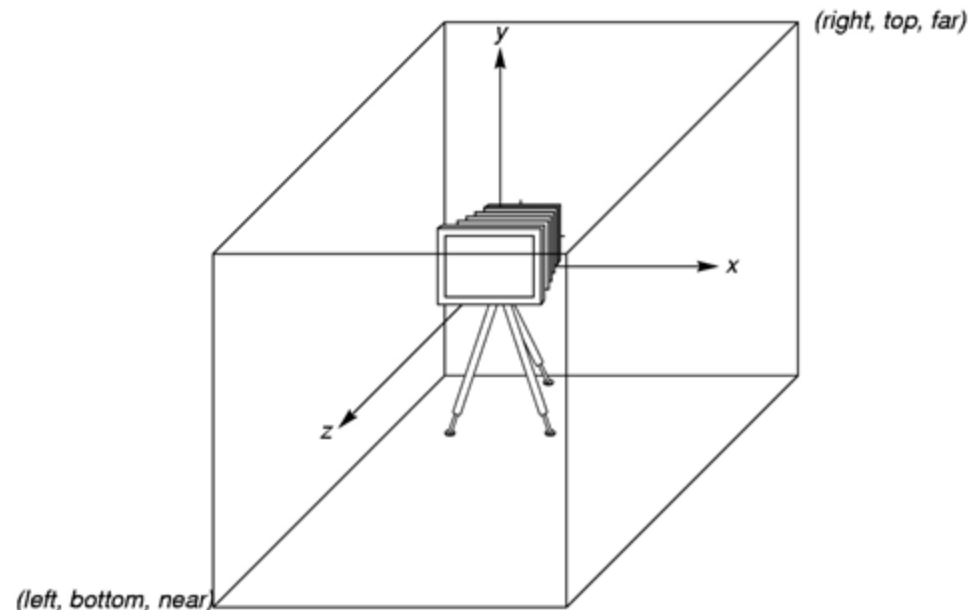
- Vertex array objects (as an extension in WebGL) and buffer objects can be set up
- Also set *clear color* and other WebGL parameters
- Also set up shaders as part of initialization
 - Create
 - Compile
- Also set up program
 - Create
 - Attach shaders
 - Link
- First let's consider a few other issues

Coordinate Systems

- The units in **points** are determined by the application and are called
 - *object*
 - *world*
 - *camera*
 - ..
coordinates
- Viewing specifications are usually in *object coordinates*
- Eventually pixels will be produced in *window coordinates*
- WebGL also uses some internal representations that usually are not visible to the application but are important in the shaders

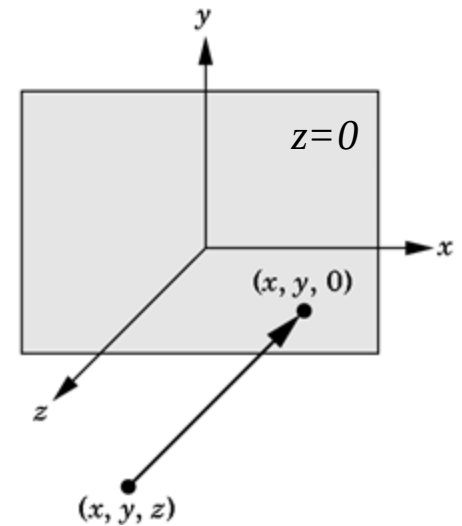
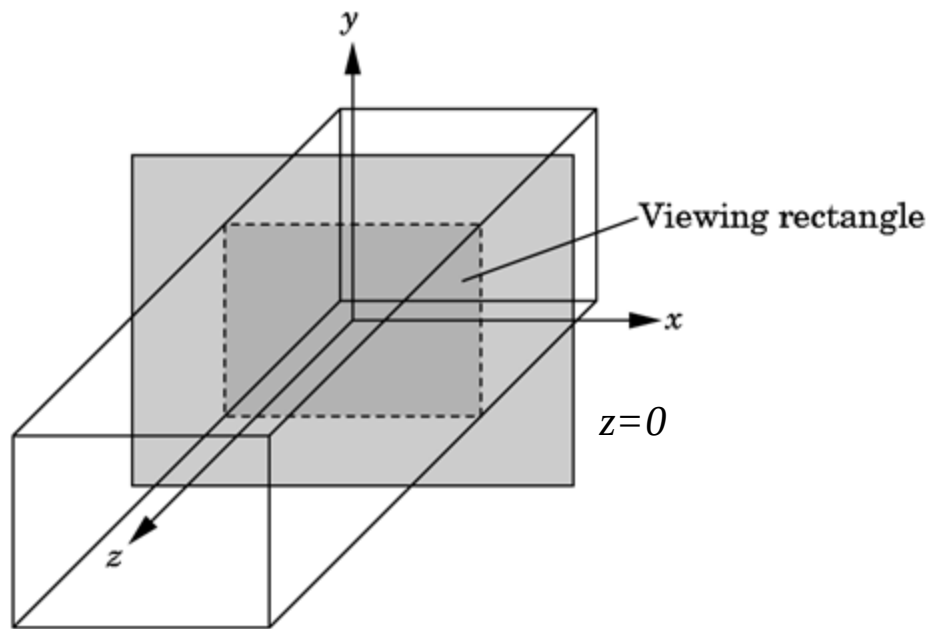
WebGL Camera

- By default: WebGL places a camera at the origin $(0,0,0)$ in object space pointing in the negative z direction
 - We are free to change this
- The default viewing volume is a box centered at the origin with sides of length 2
 - We are free to change this too



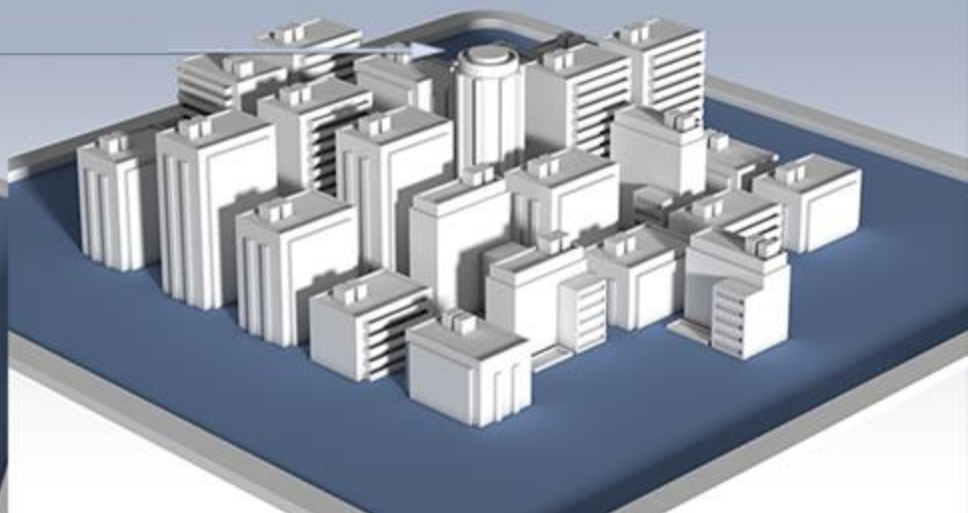
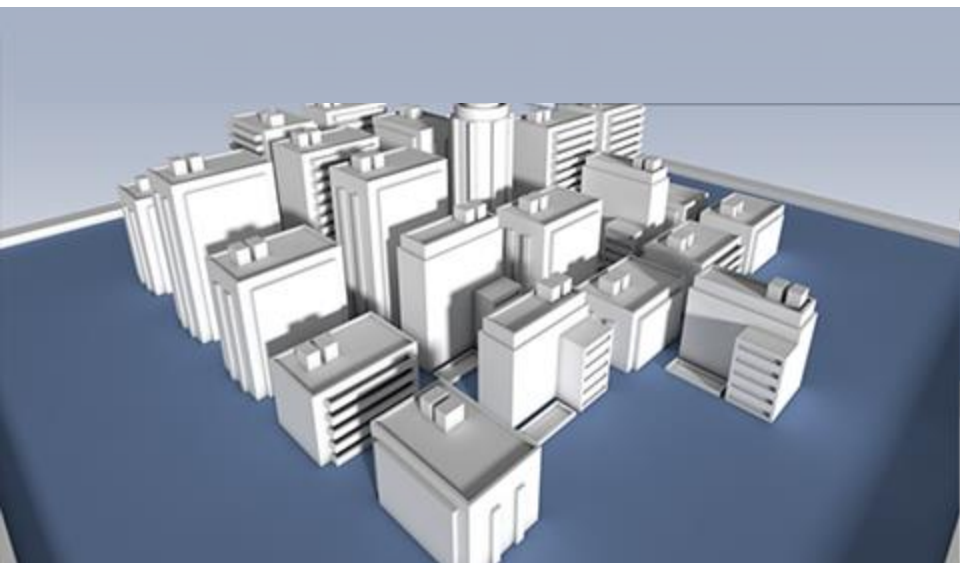
Orthographic Viewing

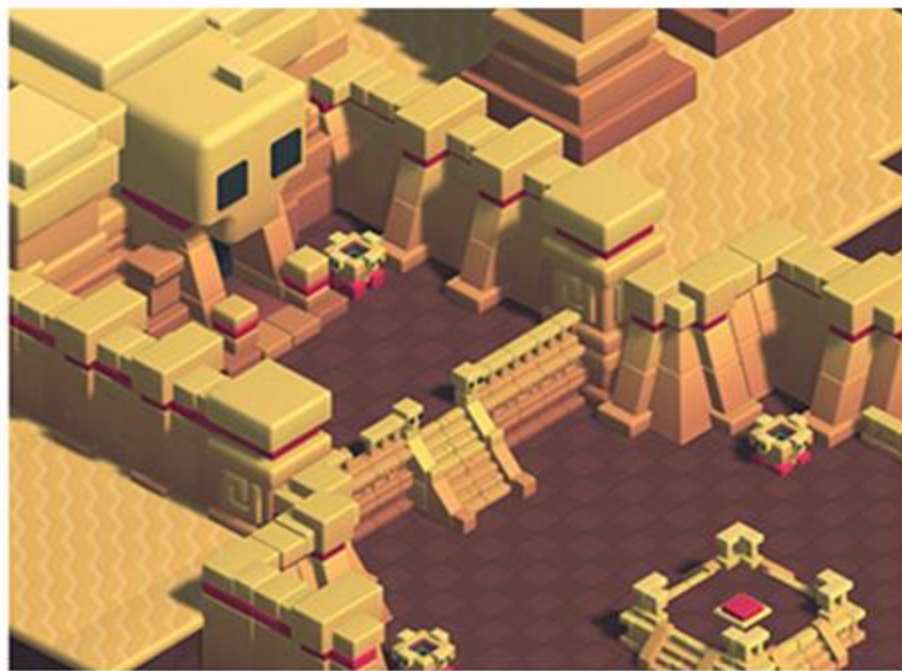
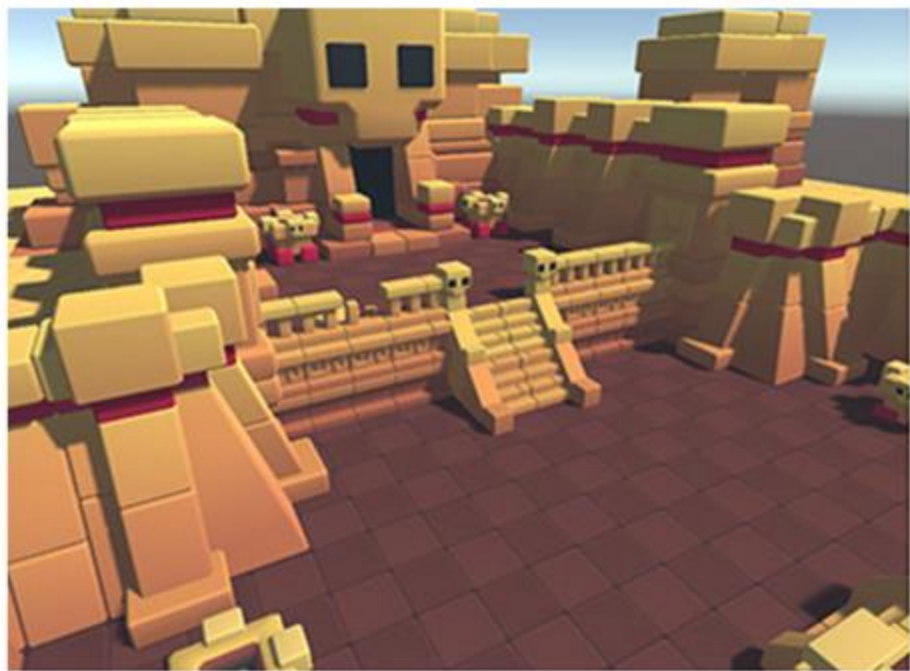
In the default orthographic view, points are projected forward along the z axis onto the plane $z=0$





0°
inf.



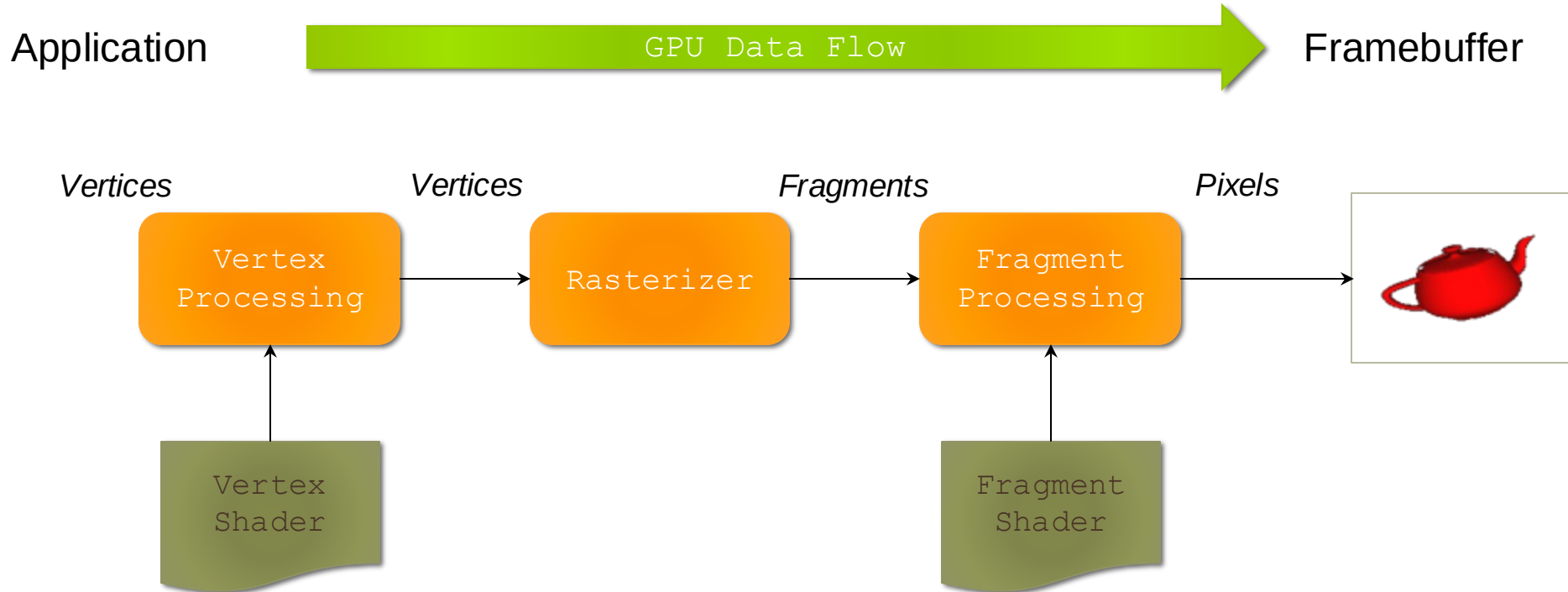


A Scene shown in Perspective mode (left) and Orthographic mode (right)

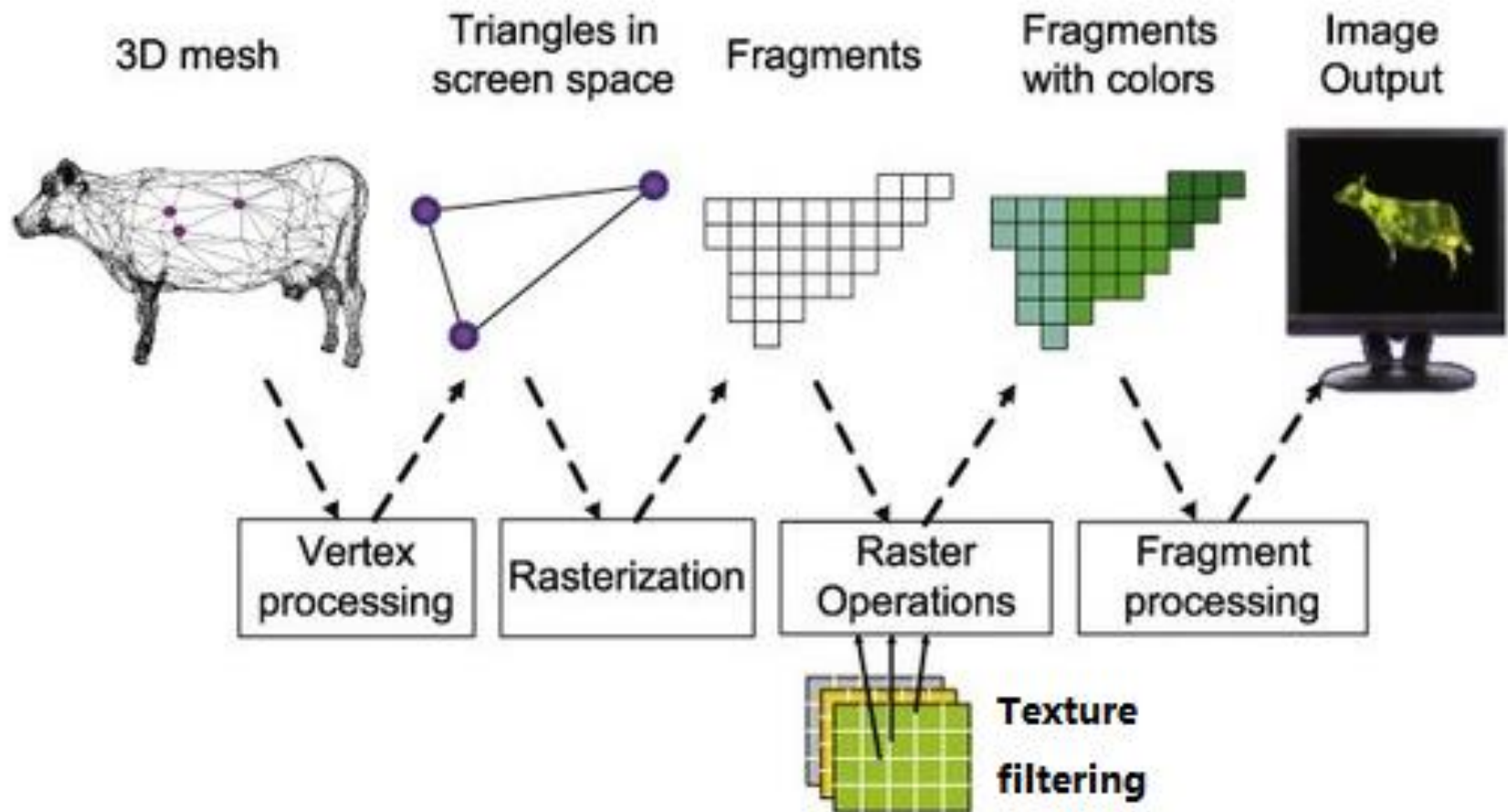
Transformations and Viewing

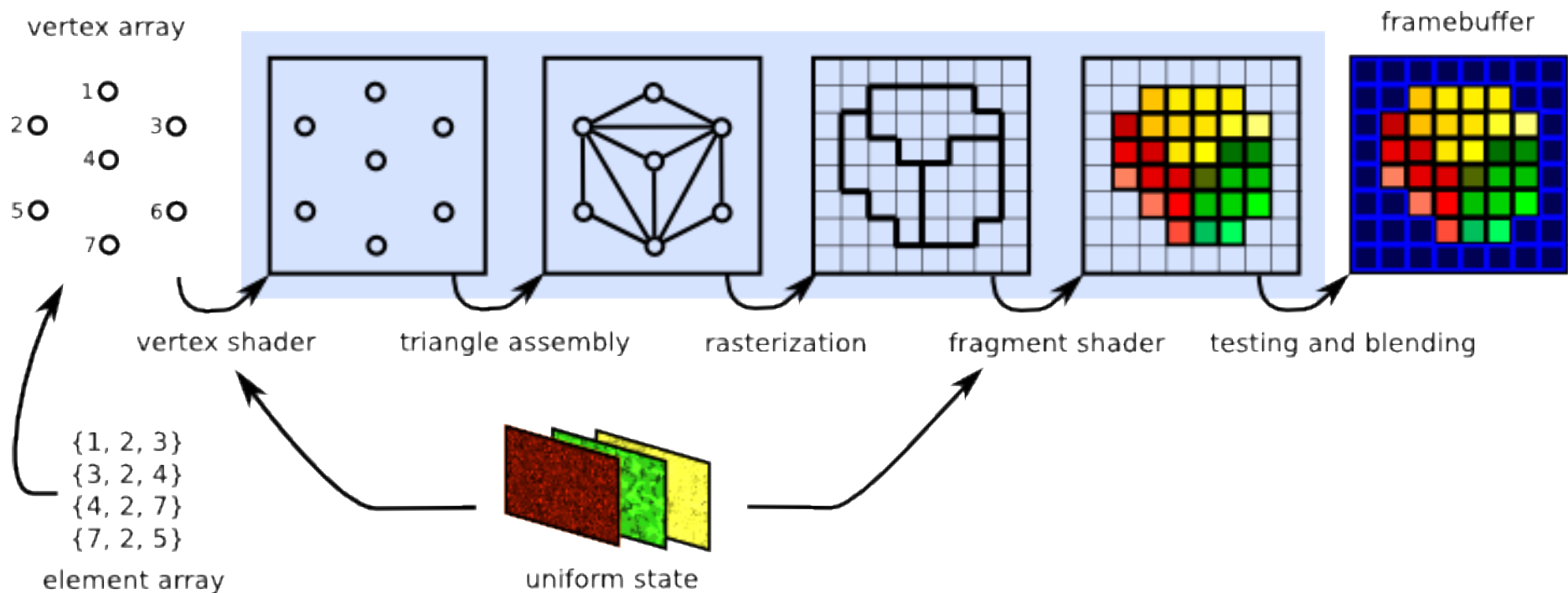
- In WebGL, projection is carried out by using a projection matrix (a transformation)
- Transformations are also used for changes in coordinate systems
- Two choices
 - Application code
 - GLSL functions

A Simplified Pipeline Model

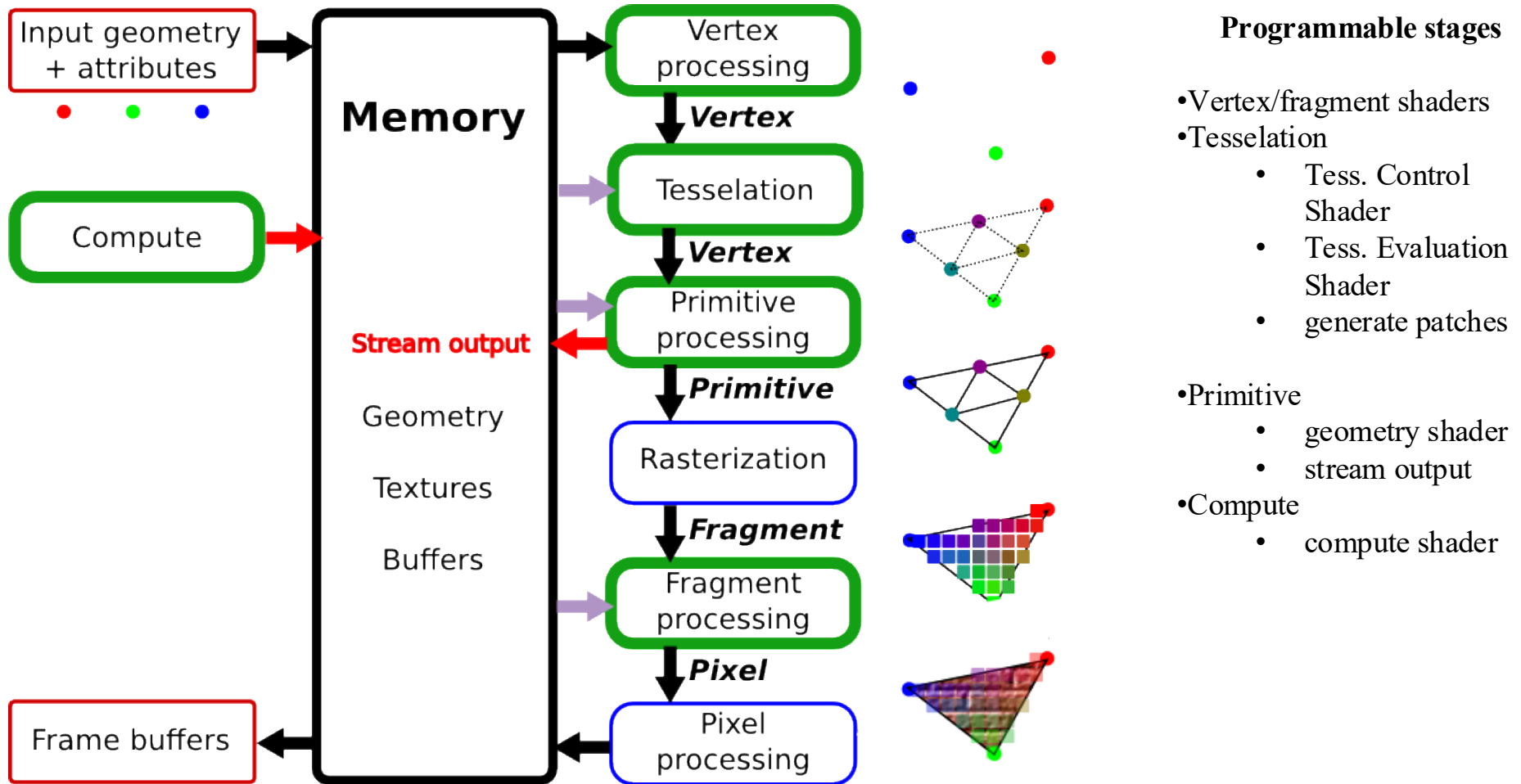


- Generally speaking, data flows from your application through the GPU to generate an image in the frame buffer.
- Your application will provide vertices to the WebGL pipeline.
 - Vertices are collections of data composed to form geometric objects.
- The vertex processing stage uses a vertex shader to process each vertex, doing any computations necessary to determine where in the frame buffer each piece of geometry should go.



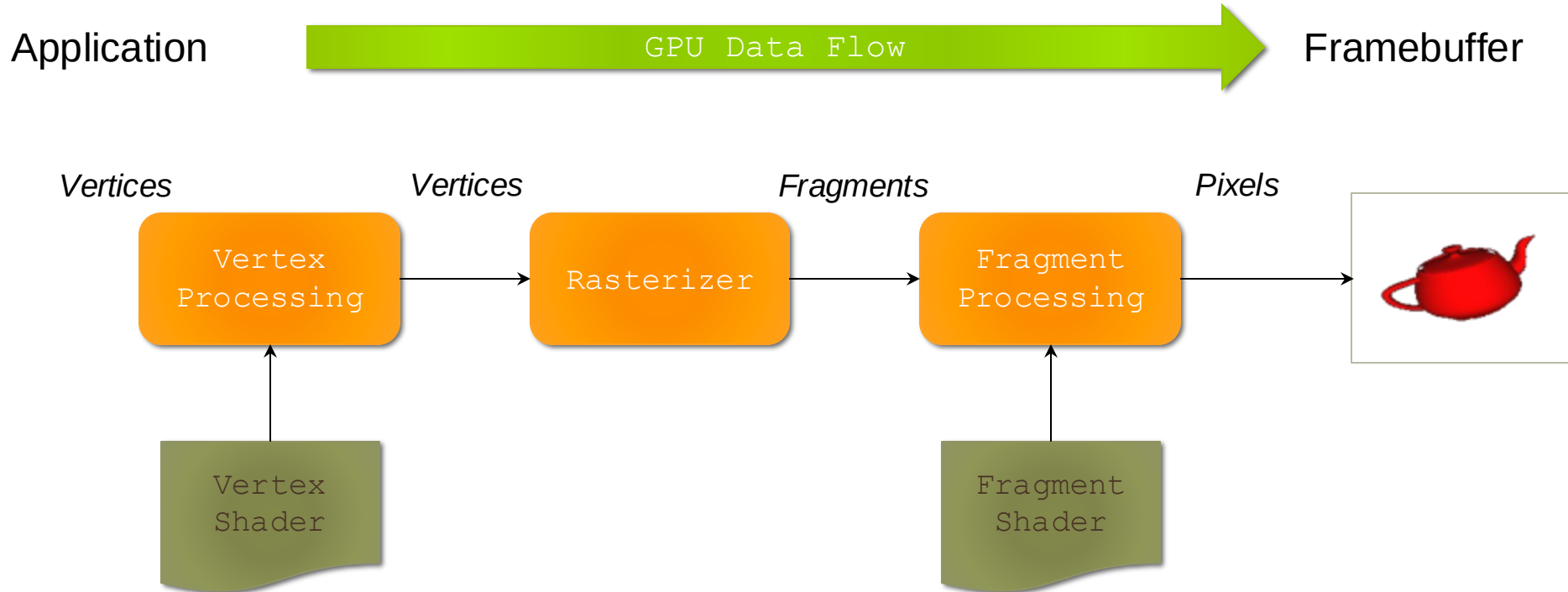


In this example, fragments are shaded (colored) during fragment processing



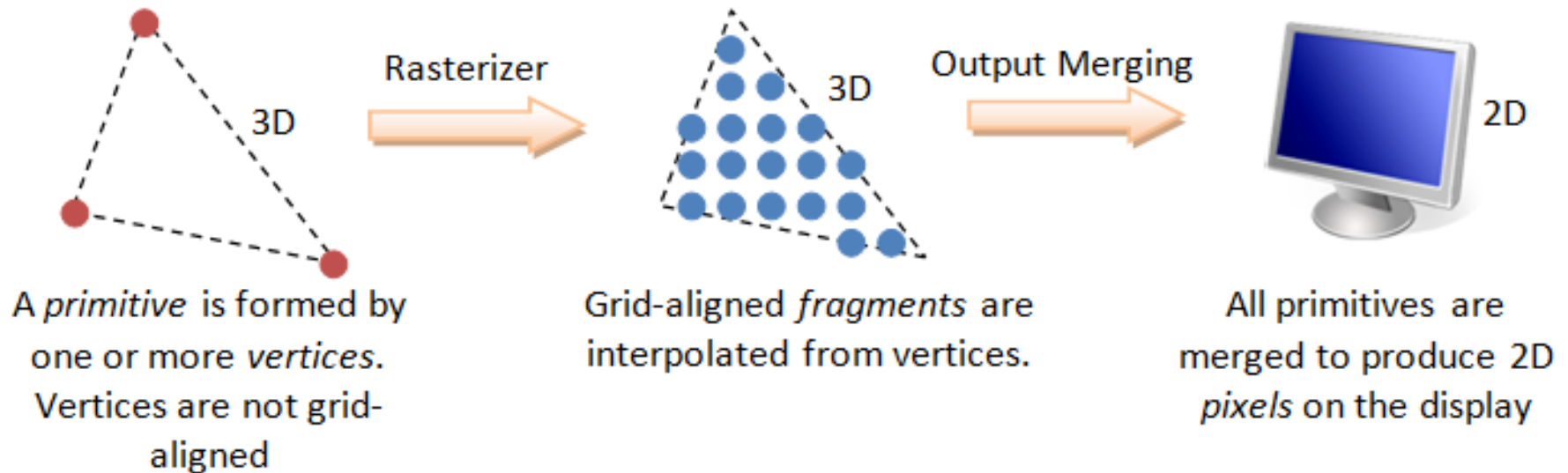
In this example, vertex colors are interpolated by the rasterizer and the interpolated colors become the temporary fragment colors. Then, the temporary colors are blended with the texture to produce the final pixel colors.

A Simplified Pipeline Model



- After all the vertices for a piece of geometry are processed, the rasterizer determines which pixels in the frame buffer are affected by the geometry. You don't have control over rasterizer.
- Extra in newer versions: The new shading stages – *tessellation* and *geometry shading* – are also used for vertex processing, but we're trying to keep this simple here so will not introduce them.

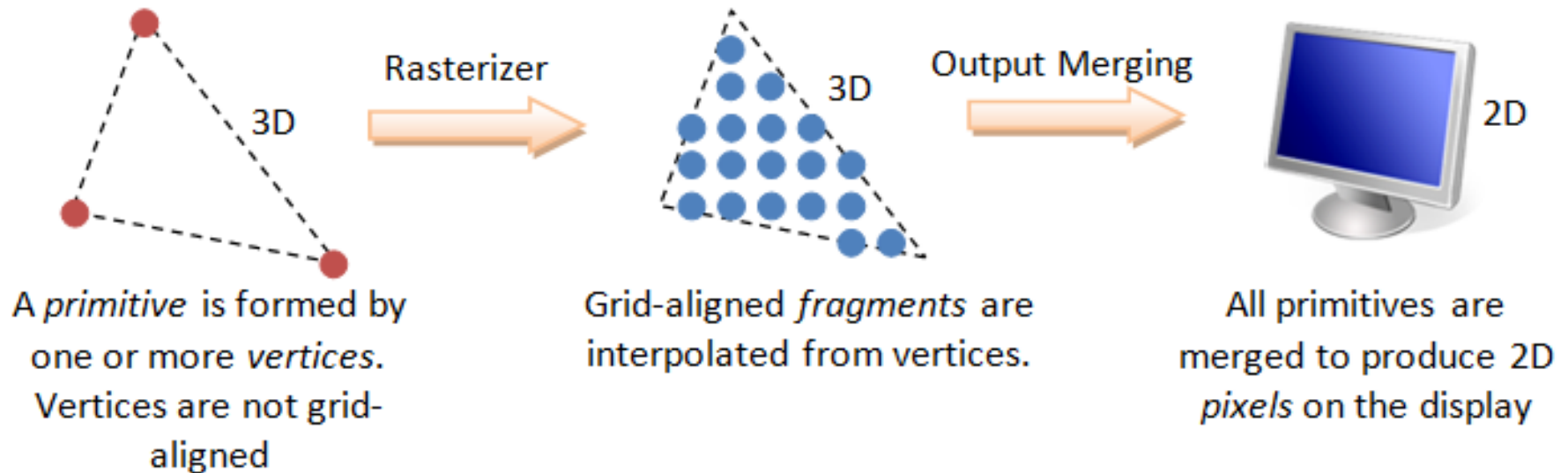
Rasterisation



Vertex, Primitives, Fragment and Pixel

- In rasterization stage, each primitive (mostly triangles) defined by a set of vertices (for triangles: 3) are raster-scanned to obtain a set of fragments enclosed within the primitive.
- Fragments can be treated as 3D pixels that are aligned with the pixel-grid.

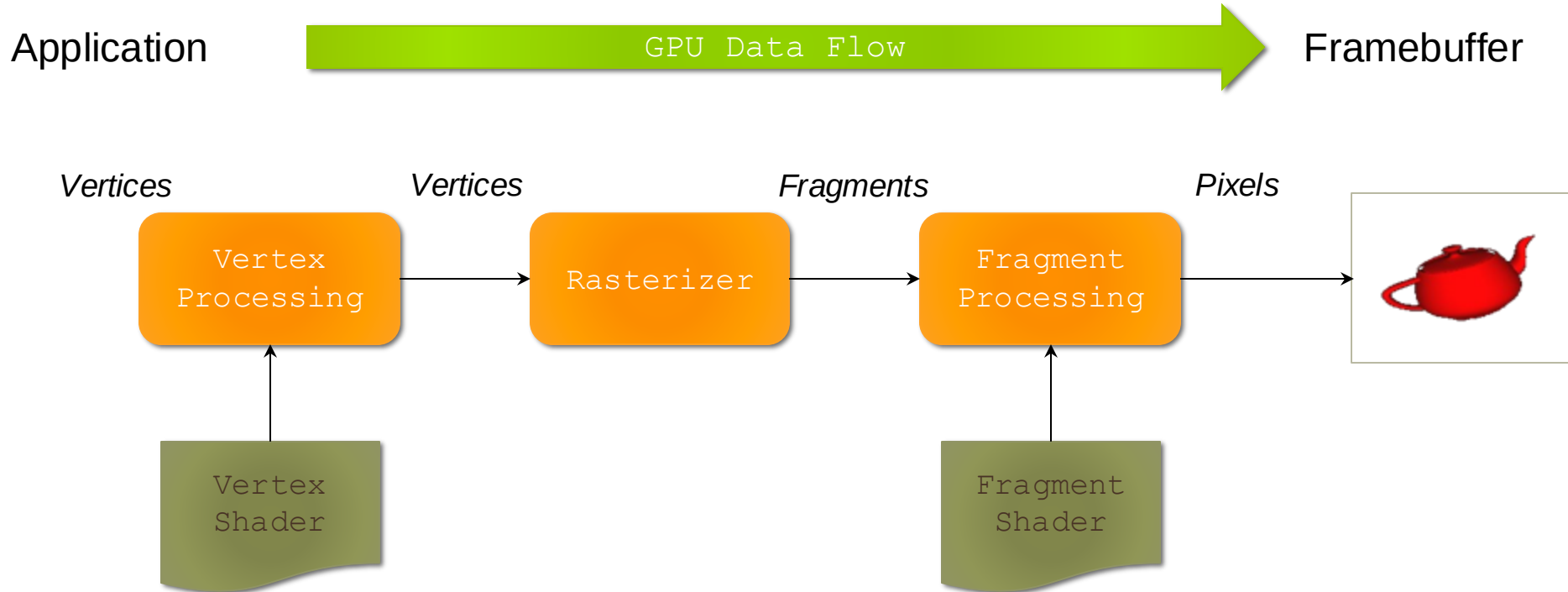
Rasterisation



Vertex, Primitives, Fragment and Pixel

- The 3D fragments, which are interpolated from the vertices, have: the same set of attributes as the vertices, such as position, color, normal vector, texture...
- The resulting 2D pixels on the display have: a position and an RGB color value.
- The rasterizer is not programmable
 - but configurable (we won't deal with the rasterizer though)

A Simplified Pipeline Model



- **fragments:** potential pixels (potential due to hidden surface removal etc.)
- fragment processing stage is employed for each pixel
- fragment shader runs to determine the final color corresponding to that pixel:
 - color them, add texture to them, do hidden surface removal..

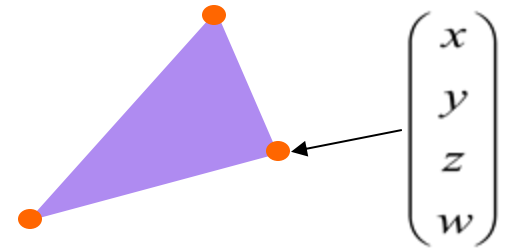
WebGL Programming in a Nutshell

Modern WebGL programs essentially go through the following steps:

1. Create shader programs
 - you'll load and create WebGL **shader programs** from shader source codes you create
 2. Create buffer objects and load data into them
 - i.e., load the data for your objects into WebGL's memory
 3. "Connect" data locations with shader variables
 - shader plumbing: WebGL needs to be told how to interpret the data in your buffer objects and associate that data with variables that you'll use in your shaders
 4. with your data initialized and shaders set up: Start/Refresh Rendering
 - rendering part is just one line that says " draw this stuff "
- **Basically**: arrange geometry data, put them onto buffer objects, send to GPU and commence rendering
 - You'll find that a few techniques for programming with modern WebGL go a long way
 - meaning: you'll reuse much of the same code over and over again
 - In fact, most programs – in terms of WebGL activity – are very repetitive
 - you'll find that most applications will merely iterate through those steps
 - Differences usually occur in
 - how objects are arranged into data to be sent to GPU
 - how objects are rendered (this is mostly handled in your shaders)

Representing Geometric Objects

- In WebGL, objects in the scene are composed of *geometric primitives*.
- Geometric primitives are represented using *vertices*.
- A vertex is a collection of generic attributes
 - Must have:
 - positional coordinates
 - Also (optional):
 - color
 - reflection information for lighting
 - texture coordinates
 - any other data associated with that point in space
 - temperature, direction of flow..
- Positions can be specified in 2, 3 or 4 dimensions but are stored in 4 dimensional *homogeneous coordinates*. (w=1 for a point, 0 for a vector)
 - while working in 2D, WebGL automatically makes z=0



Representing Geometric Objects

- Vertex data (position, color, material properties..) must be organized in WebGL server-side objects called **vertex buffer objects** (also known as **VBOs**),
 - which need to contain all of the vertex information for all of the primitives that you want to draw at one time.

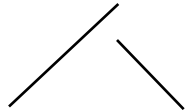
VBOs can store vertex information in almost any format

- an array-of-structures (AoS) each containing a single vertex's information,
 - or a structure-of-arrays (SoA) where all of the same “type” of data for a vertex is stored in a contiguous array, and the structure stores arrays for each attribute that a vertex can have.
-
- The data within a VBO needs to be contiguous in memory,
 - but doesn't need to be tightly packed:
 - i.e., data elements may be separated by any number of bytes, as long as the number of bytes between attributes is consistent

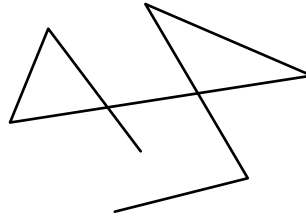
WebGL Primitives



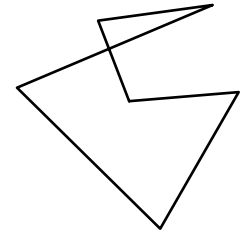
POINTS



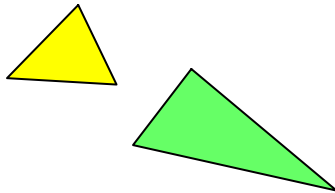
LINES



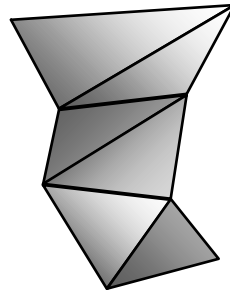
LINE_STRIP



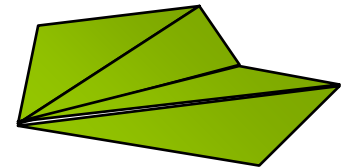
LINE_LOOP



TRIANGLES



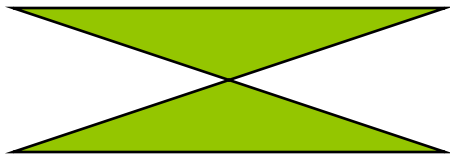
TRIANGLE_STRIP



TRIANGLE_FAN

Polygon Issues

- Modern WebGL will only display triangles. **WHY?**
 - **Simple**: edges cannot cross each other
 - **Convex**: all points on a line segment between two points in a convex polygon are also in that polygon
 - **Flat**: all vertices are in the same plane



nonsimple polygon



nonconvex polygon

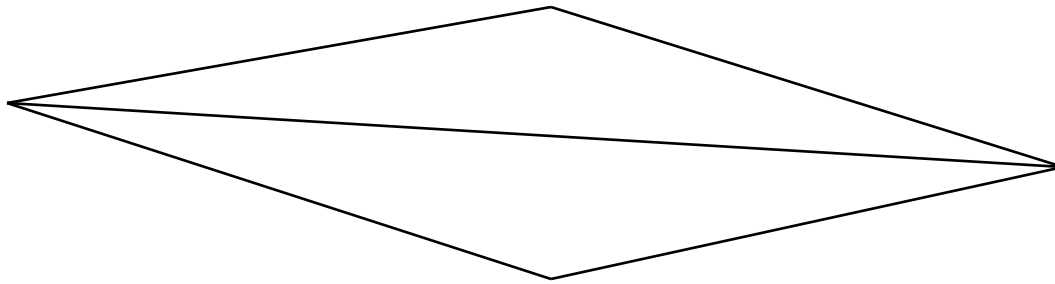
- You must either provide all geometry in triangles or the application program must tessellate polygons into triangles (triangulation)

Polygon Testing

- Conceptually simple to test for simplicity and convexity
- Time consuming
- Earlier OpenGL versions assumed both, and left testing to the application
- Present WebGL only renders triangles
- Need algorithm to triangulate an arbitrary polygon

Good and Bad Triangles

- Long thin triangles render badly



- Equilateral triangles render well
- >> Maximize minimum angle
- ***Delaunay triangulation*** for unstructured points

PROGRAMMING WITH WEBGL: **A FIRST PROGRAM**

Lecturer: Asst. Prof. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

Our First Program

- The next few slides will introduce our first serious example program,
 - one which simply displays a cube with different colors at each vertex
- We aim for simplicity in this example,
 - focusing on the WebGL techniques,
 - not on optimal performance

Our First Program

- Our example demonstrates:
 - initializing vertex data
 - organizing data for rendering
 - simple object modeling
 - building up 3D objects from geometric primitives
 - building geometric primitives from vertices

Initializing the Cube's Data

- WebGL likes to deal with hollow surfaces.
- We'll build each cube face from individual triangles.
- In order to simplify our application development, we define a few types and constants to make our code more readable and organized.
- Need to determine how many vertices are required. In order to draw the cube, we define the constant NumVertices.
 - $(6 \text{ faces}) \times (2 \text{ triangles/per face}) \times (3 \text{ vertices/per triangle})$

```
const NumVertices = 36;
```

Initializing the Cube's Data

- Additionally, as we'll see in our first shader, the WebGL shading language GLSL has a built-in type called **vec4**, which represents a vector of 4 floating-point values.
- We create the javascript objects for our application with the same semantics as that GLSL type.
- To simplify communicating with GLSL, we'll use a vec4 object (implemented in javascript) similar to GLSL's vec4 type
 - we'll also create point4 and color4 to add logical meaning

```
var point4 = vec4;  
var color4 = vec4;
```

Vertex Arrays

- Vertices can have many attributes
 - Position
 - Color
 - Texture Coordinates
 - Application data
 - ...
- A vertex array holds these data

Initializing the Cube's Data (cont'd)

- Before we can initialize our VBO, we need to stage the data so that we can load it into the VBOs that our application will use.
- In your applications, you might
 - load these data from a file,
 - or generate them on the fly >> in the code
- Our cube has two attributes per vertex
 - position
 - color
- >> We create the following two staging arrays to hold the VBO data

```
var  vPositions = [];  
var  vColors = [];
```

Note that we could organize our data in other ways

- such as with a single array with interleaved positions and colors.

Cube Data

- Vertices of a unit cube (sides=1) centered at origin
 - sides aligned with axes

```
const positions = [  
    new point4( -0.5, -0.5,  0.5, 1.0 ),  
    new point4( -0.5,  0.5,  0.5, 1.0 ),  
    new point4(  0.5,  0.5,  0.5, 1.0 ),  
    new point4(  0.5, -0.5,  0.5, 1.0 ),  
    new point4( -0.5, -0.5, -0.5, 1.0 ),  
    new point4( -0.5,  0.5, -0.5, 1.0 ),  
    new point4(  0.5,  0.5, -0.5, 1.0 ),  
    new point4(  0.5, -0.5, -0.5, 1.0 )  
];
```

```
const positions = [  
    new point4( -0.5, -0.5,  0.5, 1.0 ),  
    new point4( -0.5,  0.5,  0.5, 1.0 ),  
    new point4(  0.5,  0.5,  0.5, 1.0 ),  
    new point4(  0.5, -0.5,  0.5, 1.0 ),  
    new point4( -0.5, -0.5, -0.5, 1.0 ),  
    new point4( -0.5,  0.5, -0.5, 1.0 ),  
    new point4(  0.5,  0.5, -0.5, 1.0 ),  
    new point4(  0.5, -0.5, -0.5, 1.0 )  
];
```

- In our example we'll copy the coordinates of our cube model into a VBO for WebGL to use.
- Here we set up an array of eight coordinates for the corners of a unit cube (sides=1) centered at the origin.
- “Why do we have four coordinates for 3D data?”
- The answer is that in computer graphics, we use a fourth coordinate to represent three-dimensional coordinates to allow numerous mathematical techniques that are common operations in graphics to be done in the same way.
- This four-dimensional coordinate has a proper name: a ***homogenous coordinate***.

Cube Data (cont'd)

- We'll also set up an array of RGBA colors

```
const colors = [  
    new color4( 0.0, 0.0, 0.0, 1.0 ),    // black  
    new color4( 1.0, 0.0, 0.0, 1.0 ),    // red  
    new color4( 1.0, 1.0, 0.0, 1.0 ),    // yellow  
    new color4( 0.0, 1.0, 0.0, 1.0 ),    // green  
    new color4( 0.0, 0.0, 1.0, 1.0 ),    // blue  
    new color4( 1.0, 0.0, 1.0, 1.0 ),    // magenta  
    new color4( 1.0, 1.0, 1.0, 1.0 ),    // white  
    new color4( 0.0, 1.0, 1.0, 1.0 )    // cyan  
];
```

normalized
values

alpha channel

Generating a Cube Face from Vertices

- To simplify generating the geometry, we use a convenience function named `quad()`
 - creates two triangles for each face and assigns colors to the vertices

1 quad
= 2 triangles

```
var Index = 0; // global variable indexing into VBO arrays
```

```
function quad(a, b, c, d )
```

```
{
```

```
    vColors[Index] = colors[a]; vPositions[Index] = positions[a];  
    Index++;
```

```
    vColors[Index] = colors[b]; vPositions[Index] = positions[b];  
    Index++;
```

```
    vColors[Index] = colors[c]; vPositions[Index] = positions[c];  
    Index++;
```

1st
triangle

```
    vColors[Index] = colors[a]; vPositions[Index] = positions[a];  
    Index++;
```

```
    vColors[Index] = colors[c]; vPositions[Index] = positions[c];  
    Index++;
```

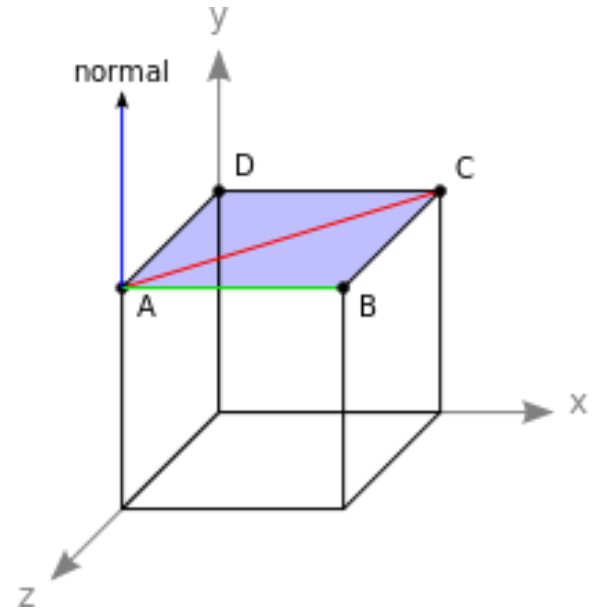
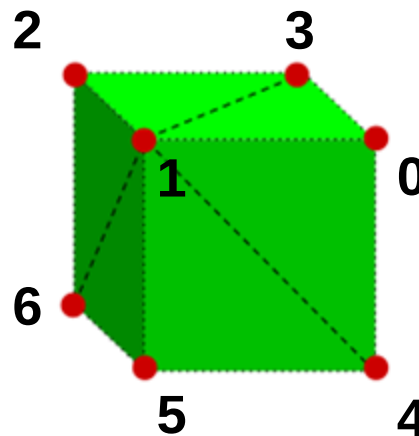
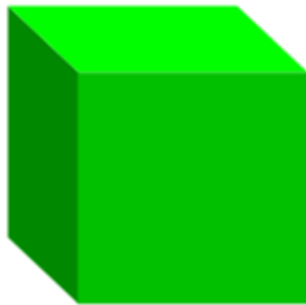
```
    vColors[Index] = colors[d]; vPositions[Index] = positions[d];  
    Index++;
```

```
}
```

2nd
triangle

Generating the Cube from Faces

- Need to generate 12 triangles for the cube
 - 36 vertices with 36 colors



```
function colorcube()
{
    quad( 1, 0, 3, 2 );
    quad( 2, 3, 7, 6 );
    quad( 3, 0, 4, 7 );
    quad( 4, 5, 6, 7 );
    quad( 5, 4, 0, 1 );
    quad( 6, 5, 1, 2 );
}
```

- order of vertex indices is important
 - *traverse using right hand rule*
 - *backface culling*

Let's remember..

- *initShaderProgram()* will create program and bind the shaders to the program
 - can be used commonly
- *loadShader()* will compile the shader
 - can be used commonly
- *initBuffer()* will create the buffer and bind the positions to it
 - can be used commonly, can have more stuff too
 - In this *colorcube* example, we do these in *main()*
- ***main()***
 - specific to the application
- application program's job:
 - describe the geometry: tell where the vertices are, how they are connected..
 - can also describe how that geometry should look: color, lighting, textures
- Key issue is that we **must form a data array** to send to GPU and then render it

```
function initShaderProgram(gl, vsSource, fsSource) {
  const vertexShader = loadShader(gl, gl.VERTEX_SHADER, vsSource);
  const fragmentShader = loadShader(gl, gl.FRAGMENT_SHADER, fsSource);
  const shaderProgram = gl.createProgram();
  gl.attachShader(shaderProgram, vertexShader);
  gl.attachShader(shaderProgram, fragmentShader);
  gl.linkProgram(shaderProgram);

  if (!gl.getProgramParameter(shaderProgram, gl.LINK_STATUS)) {
    alert('Unable to initialize the shader program: ' +
      gl.getProgramInfoLog(shaderProgram));
    return null;
  }

  return shaderProgram;
}

function loadShader(gl, type, source) {
  const shader = gl.createShader(type);

  gl.shaderSource(shader, source);
  gl.compileShader(shader);

  if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
    alert('An error occurred compiling the shaders: ' + gl.getShaderInfoLog(shader));
    gl.deleteShader(shader);
    return null;
  }

  return shader;
}
```

```
function main() {
    const canvas = document.querySelector("#glCanvas");
    const gl = canvas.getContext("webgl");

    if(!gl) {
        alert("Unable to initialize WebGL. Your browser or machine may not support it.");
        return;
    }
    colorcube();

    var positionArray = [];
    for(var i = 0; i < NumVertices; i++) {
        positionArray = positionArray.concat(vPositions[i].elements());
    }
    var colorArray = [];
    for(var i = 0; i < NumVertices; i++) {
        colorArray = colorArray.concat(vColors[i].elements());
    }
    const data = positionArray.concat(colorArray);
    gl.clearColor(0.0,0.0,0.0,1.0);
    gl.clearDepth(1.0);
    gl.enable(gl.DEPTH_TEST);
    gl.depthFunc(gl.LEQUAL);

    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
```

```
const numOfComponents = 4;
const type = gl.FLOAT;
const normalize = false; const stride = 0; var offset = 0;

const program = initShaderProgram(gl, vShader, fShader);
const buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(data), gl.STATIC_DRAW);
gl.bufferSubData(gl.ARRAY_BUFFER, 0, new Float32Array(positionArray));
gl.bufferSubData(gl.ARRAY_BUFFER, positionArray.length * 4, new
    Float32Array(colorArray));
console.log(gl.getBufferParameter(gl.ARRAY_BUFFER, gl.BUFFER_SIZE));

gl.uniformMatrix4fv(gl.getUniformLocation(program, "u_matrix"), false, matrix);

gl.enableVertexAttribArray(gl.getAttribLocation(program, "a_position"));
gl.vertexAttribPointer(gl.getAttribLocation(program, "a_position"),
    numOfComponents, type, normalize, stride, offset);

offset = positionArray.length * 4;
gl.enableVertexAttribArray(gl.getAttribLocation(program, "a_color"));
gl.vertexAttribPointer(gl.getAttribLocation(program, "a_color"),
    numOfComponents, type, normalize, stride, offset);

gl.useProgram(program);

offset = 0;
gl.drawArrays(gl.TRIANGLES, offset, NumVertices);
}
```

Storing Vertex Attributes

- Vertex data must be stored in a VBO

1. **generate** VBO names by calling `createBuffer()`
2. **bind** a specific VBO for initialization by calling

```
gl.bindBuffer( gl.ARRAY_BUFFER, ... )
```

make that buffer the “current” buffer, which means it’s the selected buffer for reading or writing data values

3. **load data** into VBO using

```
gl.bufferData( gl.ARRAY_BUFFER, ... )
```

copies data from your application (CPU) into the GPU’s memory (like `memcpy` in C).

- Reason: buses in GPU far exceed the ones in CPU >> better performance.
 - You would do the same operation if you also wanted to update data in the buffer.
-
- if you have multiple objects, each with their own VBO, you’ll likely call `gl.bindBuffer()` once per frame for each object.

VBOs in Code

- The previous sequence of calls illustrate
 - generating,
 - binding,
 - and initializinga VBO with data.
- In this example, we use a technique permitting data to be loaded into two steps, which we happen to need as our data values are in two separate arrays.
- It's noteworthy to look at the `gl.bufferData()` call; in this call,
 - we basically have WebGL allocate an array sized to our needs (our point and color arrays),
- This is akin to calling `malloc()` in C to create a buffer of uninitialized data.

VBOs in Code

- `gl.bufferData` needs to know
 - how large the data we're copying is (in this case: ? floats),
 - and a hint to inform WebGL how you expect the data to be used - either dynamically updated, or loaded once as static data.
- In this case, we're going to buffer it once and then never modify it,
- so we're going to use the `gl.STATIC_DRAW` symbolic constant as our hint.
- These hints allows our graphics card driver to handle our data more effectively,
- but don't place any actual restrictions on how we can use our data
 - we can define data as dynamic and then never update it if we really want to.
- We later load that array with our calls to `gl.bufferSubData()`,
- which allows us to replace a subsection of our array.
- This technique is also useful if you need to update data inside of a VBO at some point in the execution of your application.
- What is an alternative to this approach?
 - I could use one VBO for `vpos` and another VBO for `vcolor`

VBOs in Code

- The alternative way of doing the same thing: Using separate buffers for position and color

```
const positionBuffer = gl.createBuffer();
```

```
gl.bindBuffer( gl.ARRAY_BUFFER, positionBuffer );
```

```
gl.bufferData( gl.ARRAY_BUFFER, new  
Float32Array(vPositions), gl.STATIC_DRAW);
```

Bind positions and
colors to the buffer



```
const colorBuffer = gl.createBuffer();
```

```
gl.bindBuffer( gl.ARRAY_BUFFER, colorBuffer );
```

```
gl.bufferData( gl.ARRAY_BUFFER, new  
Float32Array(vColors), gl.STATIC_DRAW);
```

Connecting Vertex Shaders with Geometric Data

- The final step in preparing your data for processing by WebGL (i.e., sending it down for rendering) is to specify which vertex attributes you'd like issued to the graphics pipeline.
- Application vertex data enters the WebGL pipeline through the vertex shader.
- Need to connect vertex data to shader variables
 - requires knowing the attribute location
- Attribute location queried by calling `gl.getAttributeLocation()`
- While this might seem superfluous, this allows you to specify multiple collections of data, and choose which ones you'd like to use at any given time.
- Each of the attributes that we enable must be associated with an “**attribute**” variable of the currently bound vertex shader.

Vertex Array Code

- Associate shader variables with vertex arrays
 - do this after shaders are loaded
 - to complete the “plumbing” (setup) of associating our vertex data with variables in our shader programs, we need to tell WebGL where in our buffer object to find the vertex data, and which shader variable to pass the data to when we draw.

```
const vPosition =  
    gl.getAttributeLocation( program, "vPosition" );  
gl.enableVertexAttribArray( vPosition );  
gl.vertexAttribPointer( vPosition, 4, gl.FLOAT,  
    false, 0, 0);
```

```
const vColor =  
    gl.getAttributeLocation( program,"vColor" );  
gl.enableVertexAttribArray( vColor );  
gl.vertexAttribPointer( vColor, 4, gl.FLOAT,  
    false, 0, positionArray.length * 4);
```

Vertex Array Code

- In our vertex shader, we have two variables:
 - `vPosition`, and `vColor`, which we associate with the data values in our VBOs that we copied from our `vertex positions` and `colors` arrays.
- a “program” is a container object in our application that has our shaders inside of it.
 - switch between program objects to switch between sets of shaders.
- The calls to **`gl.getAttributeLocation()`** will return a compiler-generated index which we need to use to complete the connection from our data to the shader inputs.
- We also need to “turn the valve” on our data by enabling its attribute array by calling **`gl.enableVertexAttribArray()`** with the selected attribute location
 - you always use vertex positions, but you may want to change colors so you can enable and disable.

Drawing Geometric Primitives

- After getting all the “plumbing” done, it is time to draw!
- In order to initiate the rendering of primitives, you need to issue a drawing routine.
- Usually invoked in display callback
- Initiates vertex shader
- While there are many routines for this in WebGL, the simplest routine is **drawArrays()**
- For contiguous groups of vertices

```
gl.drawArrays( gl.TRIANGLES, 0, NumVertices );
```

 - what type of graphics primitive you want to draw (e.g., here we’re rendering separate triangles):
gl.TRIANGLES : “Take the vertices from the array 3 by 3 and draw each 3 as a separate triangle!”,
 - which vertex in the enabled vertex attribute arrays to start with: 0 : the first one,
 - and how many vertices to send to be drawn: **NumVertices**: “..up to the last one”
- This is the simplest way of rendering geometry in WebGL.
- You merely need to store vertex data in sequence, and then **drawArrays()** takes care of the rest.
- However, in some cases, this won’t be the most memory efficient method of doing things.
- Many geometric objects share vertices between geometric primitives, and with this method, you need to replicate the data once for each vertex.

PROGRAMMING WITH WEBGL: **SHADERS**

Lecturer: Asst. Prof. Ufuk Çelikcan

Based on the slides by: E. Angel and D. Shreiner

Vertex Shader Applications

- Fundamental things vertex shader has to do:
 - coordinate conversions,
 - put objects together
- Moving vertices
 - Morphing
 - Wave motion
 - Fractals
- Lighting
 - Cartoon shaders

Fragment Shader Applications

Per fragment lighting calculations

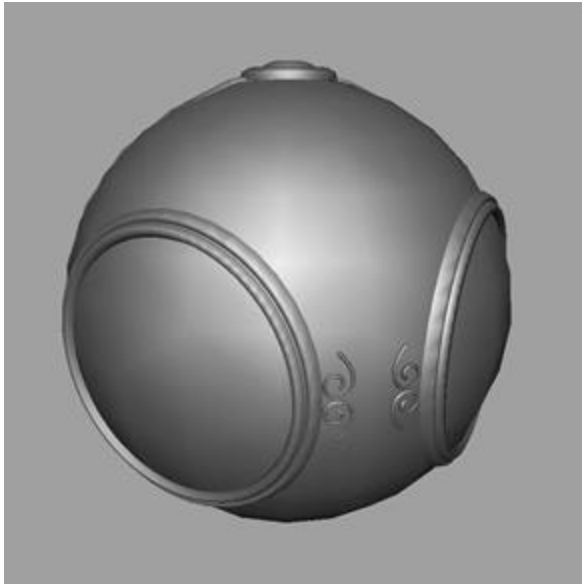


per vertex lighting



per fragment lighting

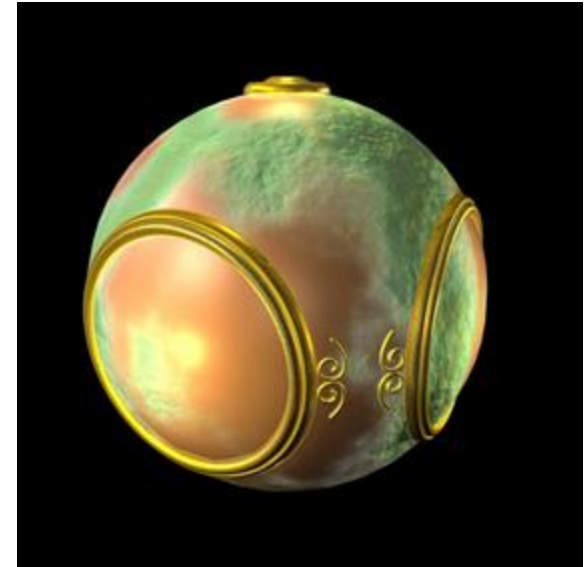
Fragment Shader Applications



smooth shading



environment
mapping



bump mapping

GLSL

- OpenGL Shading Language
- High level C-like language
 - Looks a lot like C
 - Doesn't have templates
 - Has overloaded operators
- New data types
 - Matrices
 - Vectors
 - Samplers

Data Types

- C-like scalars: `int`, `float`, `bool`
- includes vector- and matrix-based types to simplify the operations that occur often in computer graphics.
- Vector types:
 - float: `vec2`, `vec3`, `vec4`
 - Also int (ivec): `ivec2`, `ivec3`, `ivec4`
 - and boolean (bvec): `bvec2`, `bvec3`, `bvec4`
- Matrix types: `mat2`, `mat3`, `mat4`
 - Stored by columns
 - Standard referencing `m[row][column]`
- Texture sampling (to access texturemaps):
 - `sampler1D`, `sampler2D`, `sampler3D`, `samplerCube`
- C++ style constructors
 - `vec3 a = vec3(1.0, 2.0, 3.0)`
 - `vec2 b = vec2(a)`

Pointers

- There are **no pointers** in GLSL
- We can use C **structs** which can be copied back from functions
- Because matrices and vectors are basic types, they can be passed into and output from GLSL functions, e.g.
mat3 func(mat3 a)

Operators

- Standard C/C++ arithmetic and logic operators
- Overloaded operators for matrix and vector operations

```
mat4 M;  
vec4 a, b, c;
```

```
b = a*M;  
c = M*a;
```

- Overloading ensures that both $a*m$ and $m*a$ are defined
 - although they will not in general produce the same result.

Components and Swizzling

- Access vector components using either:

1. C-style array indexing: []
2. named components: `xyzw`, `rgba` or `stpq`

for regular coordinates for colors for texture coordinates



- For example:

```
vec3 v;  
v[1], v.y, v.g, v.t  all refer to the same element
```

- Vector component swizzling:

```
vec3 a, b;  
a.xy = b.yx;  
a.yz = vec2(1.0, 2.0);
```

- Matrices do not support swizzling

Qualifiers

- GLSL has many of the same qualifiers such as **const** as C/C++
- Need others due to the nature of the execution model
- Variables can change
 - Once per primitive
 - Once per vertex
 - Once per fragment
 - At any time in the application
- Vertex attributes are interpolated by the rasterizer into fragment attributes

Passing values

- call by value-return
 - Variables are copied in
 - Returned values are copied back
- 3 possibilities
 - **in**
 - **uniform**
 - **out**

In (Attribute) Qualifier

- Attribute-qualified variables can change at most once per vertex/fragment
- There are a few built in variables such as **gl_Position** (but most others have been deprecated)
- **gl_Position: required in vertex shader**
 - outputs final position of the processed vertex from vertex shader
- User defined (in application program)
 - Use **in** qualifier to get to shader
 - **in float temperature**
 - **in vec3 velocity**

Uniform Qualifier

- Variables that are constant for an entire primitive
- Can be changed in application and sent to shaders
- **Cannot be changed in shader**
- Used to pass information to shader such as the bounding box of a primitive

Out Qualifier

- Variables that are passed
 - from vertex shader to fragment shader
 - from fragment shader to framebuffer
- The ones output from the vertex shader are automatically interpolated by the rasterizer

```
out vec4 color;
```

To Sum Up the Main Qualifiers:

- **in** qualifiers: indicate the shader variable will receive data flowing into the shader,
 - from the application to vertex shader.
 - from the rasterizer to fragment shader.
- **out** qualifier which tags a variable as data output
 - where data will flow to the next shader stage, or to the framebuffer
- **uniform** qualifiers for accessing data that doesn't change across a draw operation

Examples

- **in, out**

- Copy vertex attributes and other variable into and out of shaders

```
in vec2 texCoord;  
out vec4 color;
```

- **uniform**

- shader-constant variable from application

```
uniform float time;  
uniform vec4 rotation;
```

Built-in GLSL Functions

- GLSL also provides a rich library of functions supporting common operations.
- While pretty much every vector- and matrix-related function available you can think of,
 - along with the most common mathematical functions are built into GLSL,
- there's no support for operations like reading files or printing values.
- >>In essence, shaders are data-flow engines with
 - data coming in,
 - being processed,
 - and sent out

Built-in GLSL Functions

Common Functions

T abs (T x)	absolute value
T sign (T x)	returns -1.0, 0.0, or 1.0
T floor (T x)	nearest integer $\leq x$
T ceil (T x)	nearest integer $\geq x$
T fract (T x)	$x - \text{floor}(x)$
T mod (T x, T y) T mod (T x, float y)	modulus
T min (T x, T y) T min (T x, float y)	minimum value
T max (T x, T y) T max (T x, float y)	maximum value
T clamp (T x, T minVal, T maxVal) T clamp (T x, float minVal, float maxVal)	$\min(\max(x, \text{minVal}), \text{maxVal})$
T mix (T x, T y, T a) T mix (T x, T y, float a)	linear blend of x and y
T step (T edge, T x) T step (float edge, T x)	0.0 if $x < \text{edge}$, else 1.0
T smoothstep (T edge0, T edge1, T x) T smoothstep (float edge0, float edge1, T x)	clip and smooth

Exponential Functions

T pow (T x, T y)	x^y
T exp (T x)	e^x
T log (T x)	$\ln$
T exp2 (T x)	2^x
T log2 (T x)	$\log_2$
T sqrt (T x)	square root
T inversesqrt (T x)	inverse square root

Geometric Functions

float length (T x)	length of vector
float distance (T p0, T p1)	distance between points
float dot (T x, T y)	dot product
vec3 cross (vec3 x, vec3 y)	cross product
T normalize (T x)	normalize vector to length 1
T faceforward (T N, T I, T Nref)	returns N if $\text{dot}(N_{\text{ref}}, I) < 0$, else $-N$
T reflect (T I, T N)	reflection direction $I - 2 * \text{dot}(N, I) * N$
T refract (T I, T N, float eta)	refraction vector

Angle & Trigonometry Functions

T radians (T degrees)	degrees to radians
T degrees (T radians)	radians to degrees
T sin (T angle)	sine
T cos (T angle)	cosine
T tan (T angle)	tangent
T asin (T x)	arc sine
T acos (T x)	arc cosine
T atan (T y, T x) T atan (T y_over_x)	arc tangent

GLSL Functions

- Rest of the Built-in GLSL Functions:
 - **webgl20-reference-guide.pdf**
 - under Resources
- User-defined Functions

Vertex Shader

- used for processing vertices
- A vertex shader is initiated by each vertex output of `gl.drawArrays()`
- has the required job of specifying every vertex's position in clip coordinates to the rasterizer
 - usually involves several vertex transformations.
- Optional other uses of vertex shaders
 - Moving vertices
 - Transformations
 - Per-vertex Lighting

Simple Vertex Shader

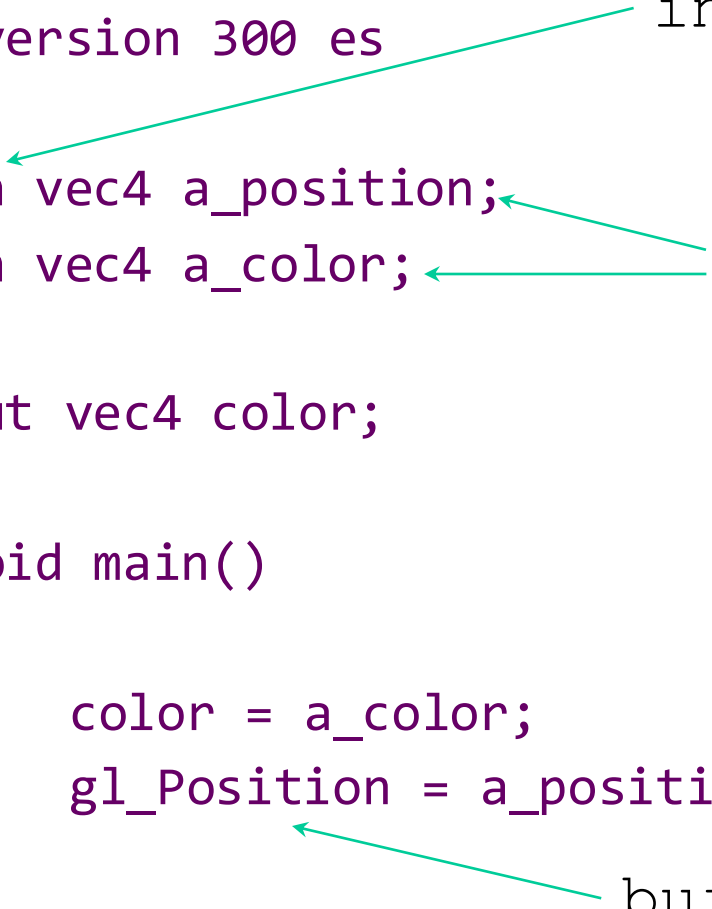
```
#version 300 es
in vec4 a_position;
in vec4 a_color;
out vec4 color;

void main()
{
    color = a_color;
    gl_Position = a_position;
}
```

input from application

must link to variable
in application

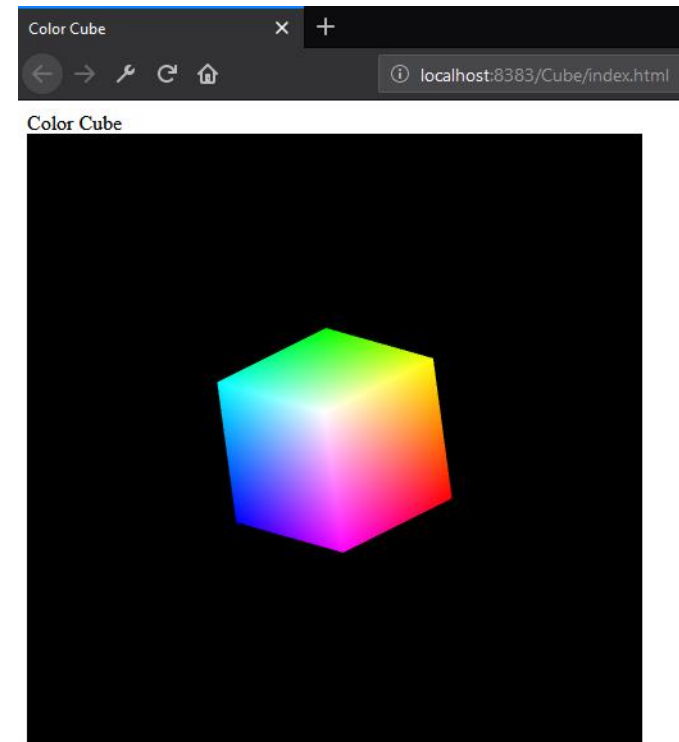
built in
variable



Simple Vertex Shader

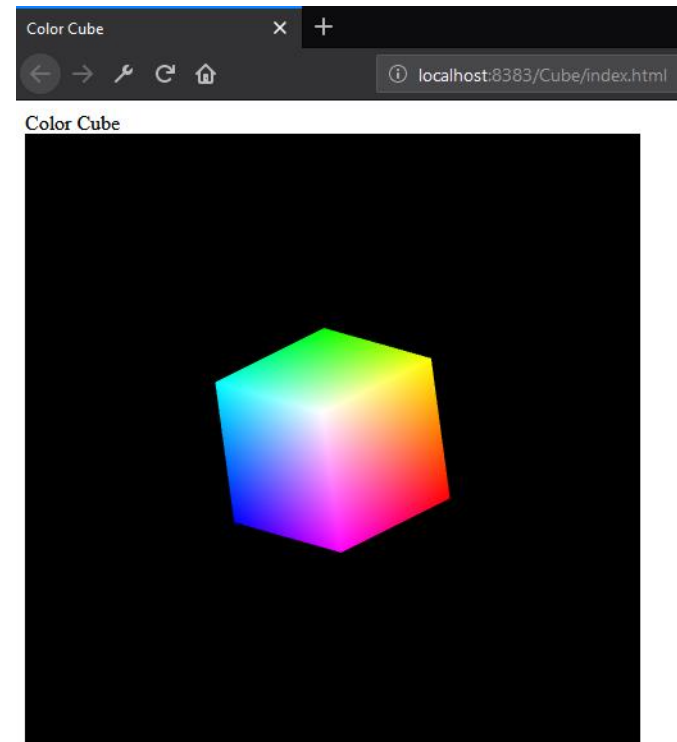
```
#version 300 es

in vec4 a_position;
in vec4 a_color;
uniform mat4 u_matrix;
out vec4 color;
void main() {
    gl_Position = u_matrix
                * a_position;
    color = a_color;
}
```

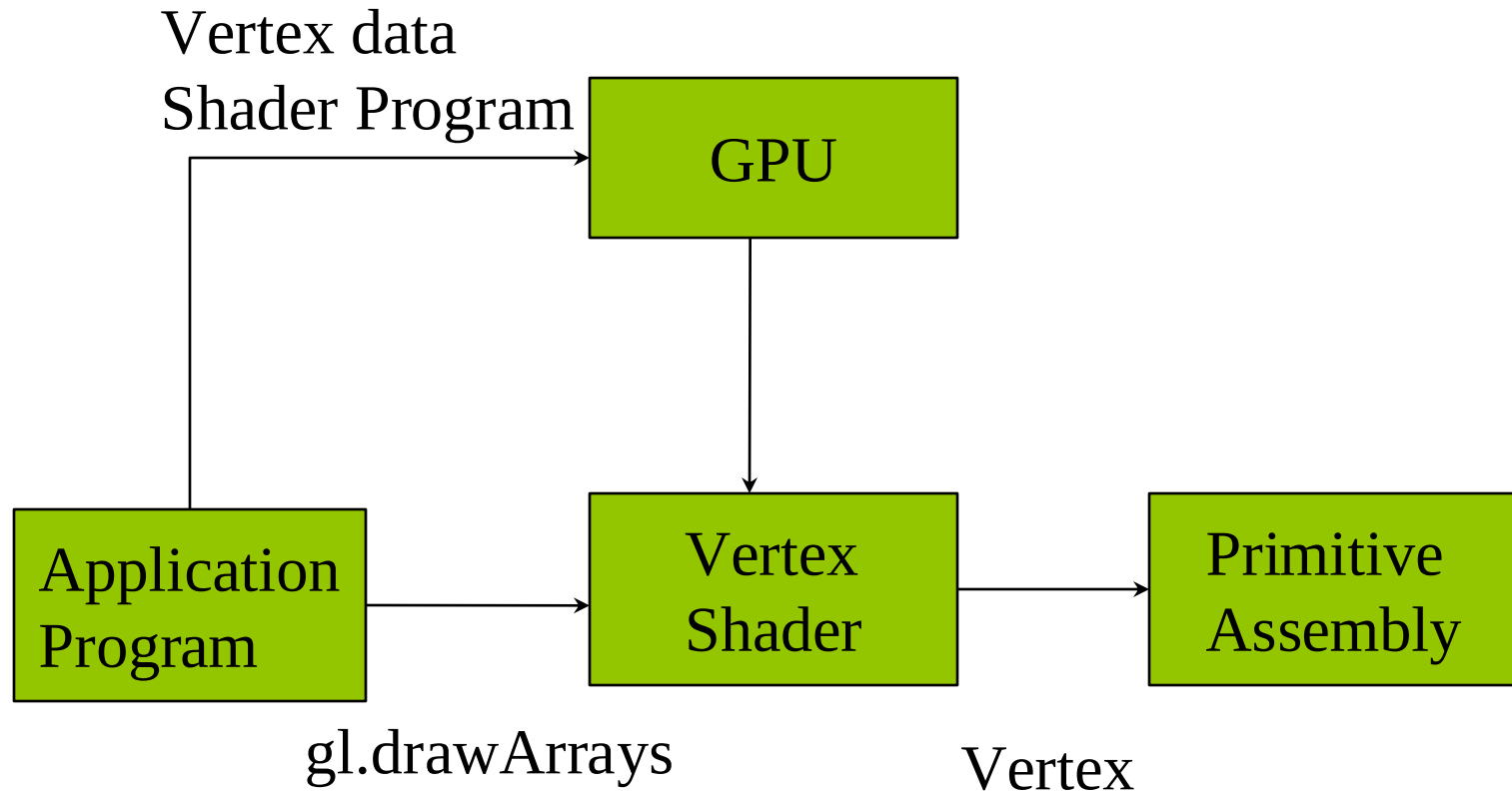


Simple Fragment Shader

```
#version 300 es
precision mediump float;
in vec4 color;
out vec4 fcolor;
void main() {
    fcolor = color;
}
```



Execution Model



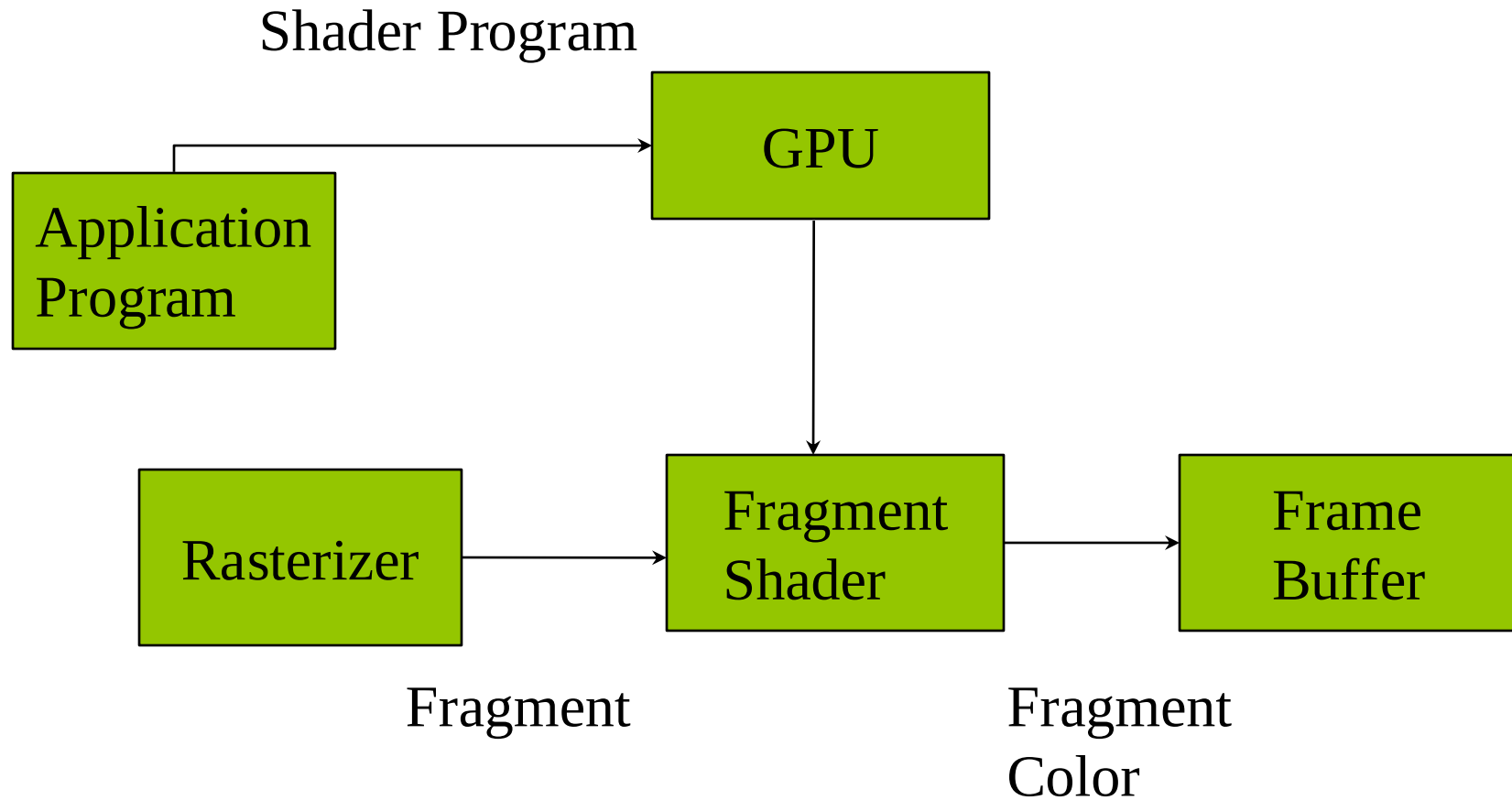
Fragment Shaders

- The final shading stage that WebGL supports is *fragment shading* which allows an application per-pixel-location control over the color that may be written to that location.
- A fragment shader's executed for each fragment:
“potential” pixel
 - fragments still need to pass several tests before making it to the framebuffer
- There are lots of effects we can do in fragment shaders
 - Per-fragment lighting
 - Texture and Bump Mapping
 - Environment (Reflection) Maps

The Simplest Fragment Shader

```
#version 300 es
precision mediump float;
in vec4 color; // comes from vertex shader
out vec4 fcolor; //output to framebuffer
void main()
{
    fcolor= color;
}
```

Execution Model



Example: Vertex Shader

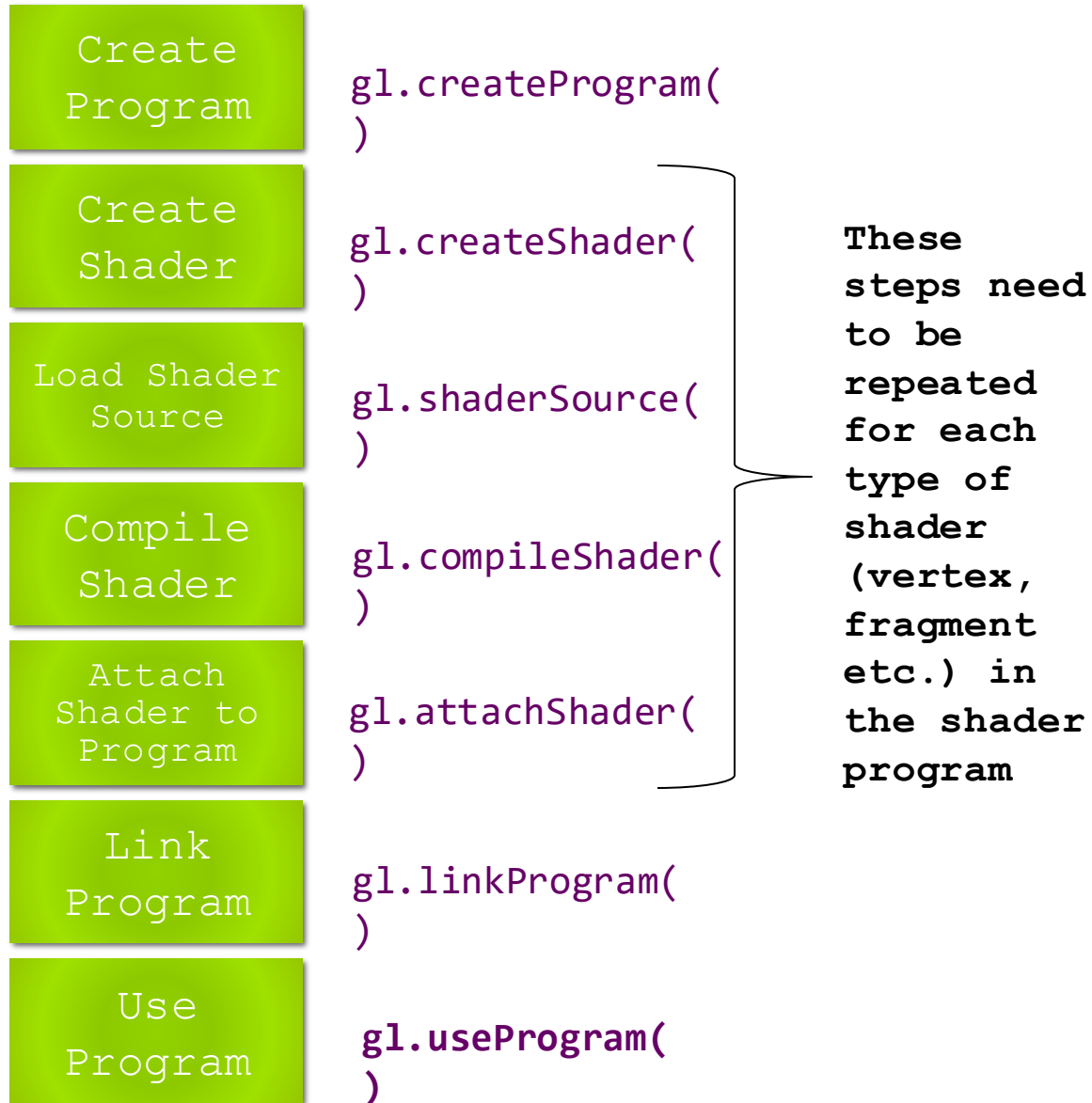
```
in vec4 vPosition;  
const vec4 red = vec4(1.0, 0.0, 0.0, 1.0);  
out vec3 color_out;  
  
void main(void)  
{  
    gl_Position = vPosition;  
    color_out = red;  
}
```

Required Fragment Shader

```
in vec3 color_out;  
out vec3 fcolor;  
  
void main(void)  
{  
    fcolor = color_out;  
}
```

TMI: Getting Your Shaders into WebGL

- Shaders need to be compiled and linked to form an executable shader program
- WebGL provides the compiler and linker. These are accessible from your application.
- A program must contain
 - **vertex** and **fragment** shaders
 - other shaders are optional



TMI: Getting Your Shaders into WebGL

- **gl.createProgram():** allocating a little disk space
- **gl.createShader():** like creating a source code file
- **gl.shaderSource():** populate that source code file.
- **gl.compileShader():** compiling that source file.
- **gl.attachShader():** like putting an object file online.
- **gl.linkProgram():** takes those object files,
 - makes sure
 - the variables match up,
 - that data can flow from vShader to fShader.
- **gl.useProgram():** linkediginiz programi kullanacagini webgl'e belirtiyorsunuz.
- uygulama sudur: oyunda vs.de bu stepe kadar olan herseyi programin basinda yaparsiniz,
- sonra gerektiginde glUseProgram diyip istediginiz shader programi istediniz zaman kullanirsiniz.

Getting Your Shaders into WebGL

- Just as with regular programs:
 - a syntax error from the compilation stage,
 - or a missing symbol from the linker stagecould prevent the successful generation of an executable program.
- Can make use of routines for verifying the results of the compilation and link stages of the compilation process.

A Simpler Way

- Angel&Shrenier created a routine for the students to make it easier to load shaders
 - available at course website

```
GLuint InitShaders( const char* vFile,  
                   const char* fFile );
```

- `InitShaders` takes two filenames
 - `vFile` path to the vertex shader file
 - `fFile` for the fragment shader file
- It implements the shader compilation and linking process shown on the previous slides.
- Also does full error checking. Fails if shaders don't compile, or program doesn't link
- The value returned from `InitShaders()` will be a valid GLSL program id that you can pass into `glUseProgram()`.

Let's take a look

<http://interactivecomputergraphics.com/>

specifically

<http://interactivecomputergraphics.com/8E/Code%20update/04/cubev.html>

Associating Shader Variables and Data

- Need to associate a shader variable with a WebGL data source
 - vertex shader attributes → app vertex attributes
 - shader uniforms → app-provided uniform values
- Each time a vertex shader executes, it's presented with new data for a single vertex. Likewise for fragment, and the other shader stages in the pipeline.
- The number of executions of a particular shader depends on how much data was associated with the draw call that started the pipeline
 - if you call `drawArrays()` specifying 100 vertices, your vertex shader will be called 100 times, each time with a different vertex.
- WebGL relates shader variables to indices for the app to set.
 - WebGL maps the names we provide as strings to integer indices. Thus, it gains speed during processing. Instead of searching for a long time to see if this string exists here, it runs its job directly with the index number.

Determining Locations After Linking

- Assumes you already know the variables' names

```
var loc = gl.getAttribLocation( program, "name" );
```

```
var loc = gl.getUniformLocation( program, "name" );
```

- Here, the value returned by the "program" is the **loc**: an index number
 - If -1 received: this indicates that the written shader program is not compiled or the variable you are looking for is not found.
 - "Name" is of course the name of the variable.
-
- If you don't know the variables in a shader (if, for instance, you're writing a library that accepts shaders), you can find out the shader variables by using the **gl.getActiveAttrib()** function.

```
const numAttribs = gl.getProgramParameter(program, gl.ACTIVE_ATTRIBUTES);
for (let i = 0; i < numAttribs; ++i) {
  const info = gl.getActiveAttrib(program, i);
  console.log("name:", info.name, "type:", info.type, "size:", info.size);
}
```

Initializing Uniform Variable Values

- Uniform Variables

`uniform4f( loc, x, y, z, w );`

index
↓

- 4f vector: needs 4 float values
- For setting a vector type, you'll use one of the **uniform*()** variants,
- and for matrices you'll use a **uniformMatrix *()** form.

requestAnimationFrame()

- For animations in WebGL, it is enough to use javascript built-in function
 - requestAnimationFrame(render);

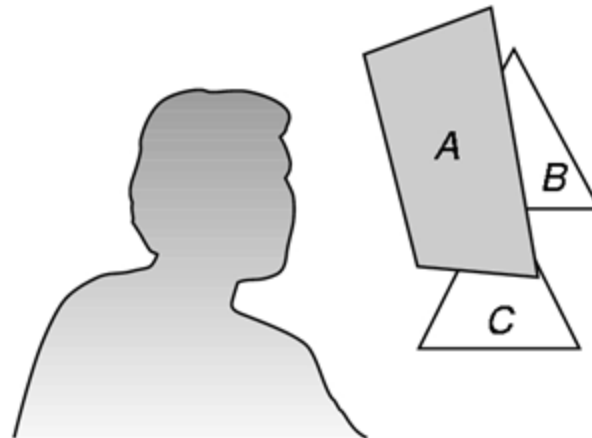
```
function render()
{
    // recompute display
    gl.clear( gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
    if(toggle) theta[axis] += 2.0;
    gl.uniform3fv(thetaLoc, theta);

    gl.drawArrays( gl.TRIANGLES, 0, NumVertices );

    requestAnimationFrame(render);
}
```

Hidden-Surface Removal

- Render only visible surfaces
- WebGL uses a *hidden-surface* method called the **z-buffer algorithm** that saves depth information as objects are rendered so that only the front objects appear in the image



Using the z-buffer algorithm

- The algorithm uses an extra buffer, namely the z-buffer (or depth buffer), to store depth information as geometry travels down the pipeline.
- It must be
 - Enabled in `main`
 - `gl.enable(gl.DEPTH_TEST)`
 - Cleared in the display callback
 - `gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT)`

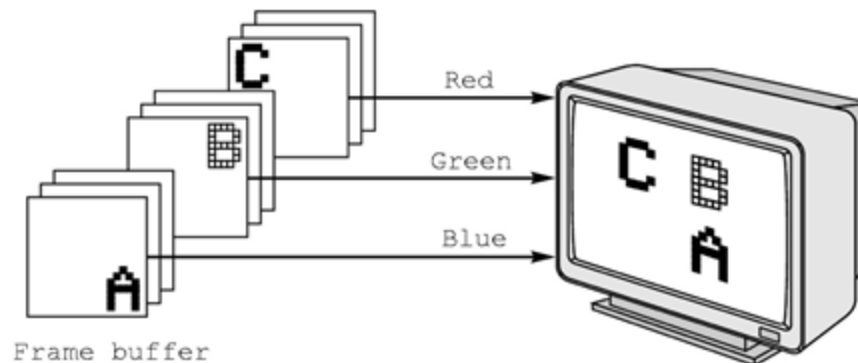
PROGRAMMING WITH WEBGL: COLORS

Lecturer: Dr. Ufuk Celikcan

Based on the slides by: E. Angel and D. Shreiner

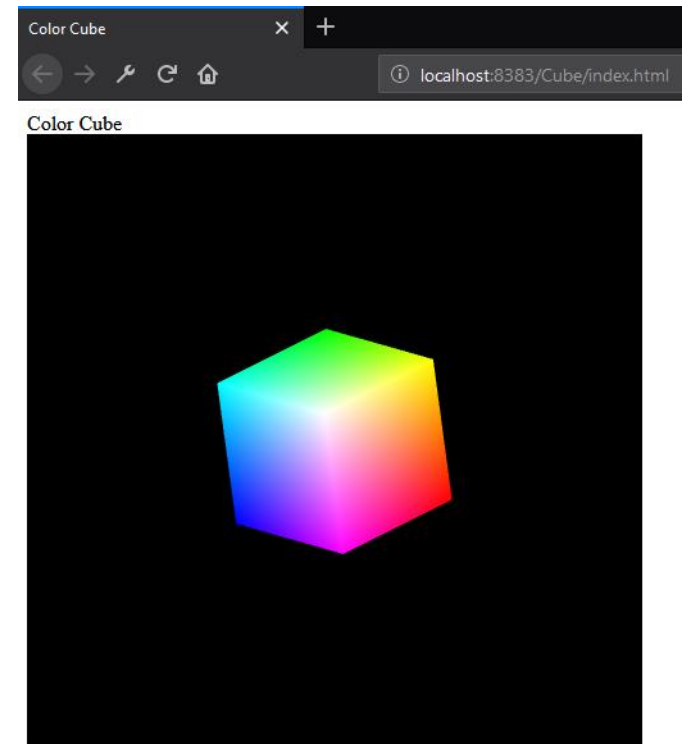
RGB color

- Each color component is stored separately in the frame buffer
- Usually 8 bits per component in buffer
- Color values can range from 0.0 (none) to 1.0 (all) using floats or over the range from 0 to 255 using unsigned bytels



Smooth Color

- Default is *smooth* shading
 - WebGL interpolates vertex colors across visible polygons
- Alternative is *flat shading*
 - Color of first vertex determines fill color
 - Handle in shader



Setting Colors

- Colors are ultimately set in the fragment shader but can be determined in either shader or in the application
 - *Application color*: pass to vertex shader as a uniform variable or as a vertex attribute
 - *Vertex shader color*: pass to fragment shader as varying variable
 - *Fragment color*: can alter via shader code

INPUT AND INTERACTION

Lecturer: Dr. Ufuk Celikcan

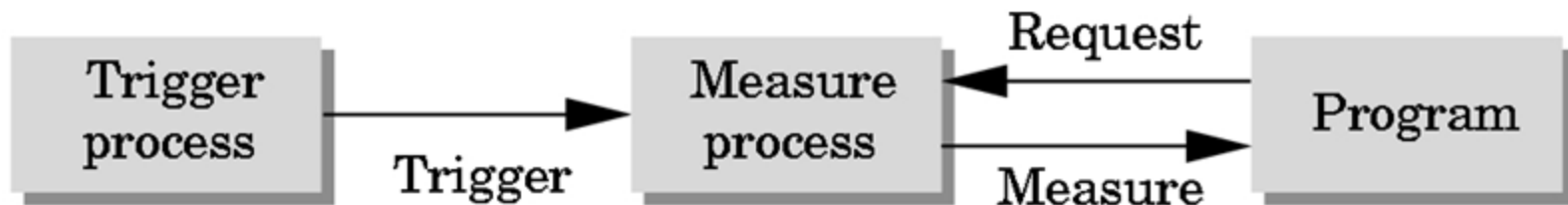
Based on the slides by: E. Angel and D. Shreiner

Input Modes

- Input devices contain a *trigger* which can be used to send a signal to the operating system
 - Button on mouse
 - Pressing or releasing a key
- When triggered, input devices return information (their *measure*) to the system
 - Mouse returns position information
 - Keyboard returns ASCII code

Request Mode

- Input provided to program only when user triggers the device
- Typical of keyboard input
 - Can erase (backspace), edit, correct until enter (return) key (the trigger) is depressed



Event Mode

- Most systems have more than one input device, each of which can be triggered at an arbitrary time by a user
- Each trigger generates an *event* whose measure is put in an *event queue* which can be examined by the user program



Event Types

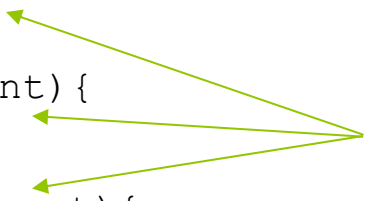
- Mouse: click one or more buttons
- Motion: move mouse
- Keyboard: press or release a key

Callbacks

- Programming interface for event-driven input
- Define a *callback function* for each type of event the graphics system recognizes
- This user-supplied function is executed when the event occurs
- `object.addEventListener("mousedown", myScript);`

The Mouse Events

```
canvas.addEventListener("mousedown", function(event){
    redraw = true; });
canvas.addEventListener("mouseup", function(event){
    redraw = false; });
canvas.addEventListener("mousemove", function(event){
    if(redraw) {
        gl.bindBuffer(gl.ARRAY_BUFFER, vBuffer);
        //normalize to range -1 to 1 (to NDC) and invert Y coordinate
        var t = vec2(2*event.clientX/canvas.width-1,
                    2*(canvas.height-event.clientY)/canvas.height-1);
        gl.bufferSubData(gl.ARRAY_BUFFER, 8*index, flatten(t));
        gl.bindBuffer(gl.ARRAY_BUFFER, cBuffer);
        t = vec4(colors[(index)%7]);
        gl.bufferSubData(gl.ARRAY_BUFFER, 16*index, flatten(t));
        index++;
    }
} );
```



mouse callback
functions
embedded
inside event
listeners

see <https://www.interactivecomputergraphics.com/Code/03/squarem.html>
for this **SquareM Example**
and <https://www.interactivecomputergraphics.com/Code/03/>
for others

flatten(..)

Copies flat or nested arrays into a typed array, where data can be:

- a nested array s.t. [[x, y], [x, y]]
- a flat array s.t. [x, y, z, x, y, z]
- a typed array s.t. new Float32Array([x, y])

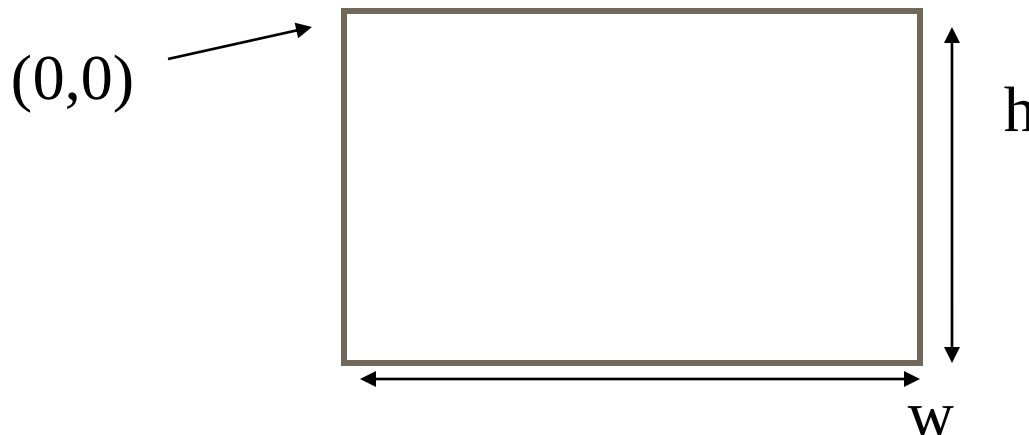
```
var positions = [ [x1, y1], [x2, y2], [x3, y3] ]
```

```
flatten(positions)
```

```
//=> new Float32Array([ x1, y1, x2, y2, x3, y3 ])
```

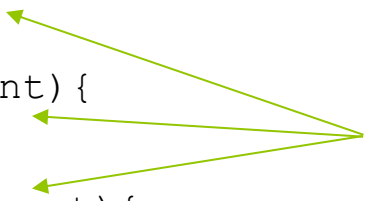
Positioning

- The position in the screen window is usually measured in pixels from the top-left corner
 - Consequence of refresh done from top to bottom
- WebGL uses a coordinate system with origin at the bottom left
 - **>> Must invert y coordinate returned by callback by height of window**
 - $y = h - y;$



The Mouse Events

```
canvas.addEventListener("mousedown", function(event){
    redraw = true; });
canvas.addEventListener("mouseup", function(event){
    redraw = false; });
canvas.addEventListener("mousemove", function(event){
    if(redraw) {
        gl.bindBuffer(gl.ARRAY_BUFFER, vBuffer);
        //normalize to range -1 to 1 (to NDC) and invert Y coordinate
        var t = vec2(2*event.clientX/canvas.width-1,
                    2*(canvas.height-event.clientY)/canvas.height-1);
        gl.bufferSubData(gl.ARRAY_BUFFER, 8*index, flatten(t));
        gl.bindBuffer(gl.ARRAY_BUFFER, cBuffer);
        t = vec4(colors[(index)%7]);
        gl.bufferSubData(gl.ARRAY_BUFFER, 16*index, flatten(t));
        index++;
    }
} );
```



mouse callback
functions
embedded
inside event
listeners

see <https://www.interactivecomputergraphics.com/Code/03/squarem.html>
for this **SquareM Example**
and <https://www.interactivecomputergraphics.com/Code/03/>
for others

Example: Mouse Callback Functions Implemented Separately

- The onmouseover and onmouseout events

- The onmouse and onmouseout events can be used to trigger a function **when the user mouses over or out of:**

```
<div onmouseover="mOver(this)" onmouseout="mOut(this)" style="background-color:#D94A38; width:120px; height:20px; padding:40px;">Mouse Over Me</div>

<script>
    function mOver(obj) { obj.innerHTML = "Thank You" }
    function mOut(obj) { obj.innerHTML = "Mouse Over Me" }
</script>
```

- The onmousedown, onmouseup and onclick events

- The onmousedown, onmouseup and onclick events are all **part of a mouse-click**

```
<div onmousedown="mDown(this)" onmouseup="mUp(this)"
style="background-color:#D94A38;width:90px;height:20px;padding:40px;">
Click Me</div>

<script>
    function mDown(obj) {
        obj.style.backgroundColor = "#1ec5e5";
        obj.innerHTML = "Release Me";
    }
    function mUp(obj) {
        obj.style.backgroundColor="#D94A38";
        obj.innerHTML="Thank You";
    }
</script>
```

Using the keyboard

- **onkeyup event**

```
<input type="text" id="fname" onkeyup="myFunction()">
<script>
function myFunction() {
  var x = document.getElementById("fname");
  x.value = x.value.toUpperCase();
}
</script>
```

- **onkeydown event**

```
<input type="text" onkeydown="myFunction()">
<script>
  function myFunction() {
    alert("You pressed a key inside the input field");
  }
</script>
```

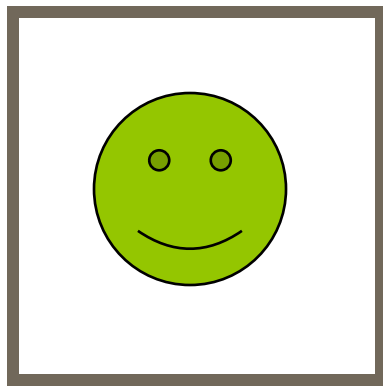
- **onkeypress event**

```
<input type="text" onkeypress="myFunction()">
<script>
function myFunction() {
  alert("You pressed a key inside the input field");
}
</script>
```

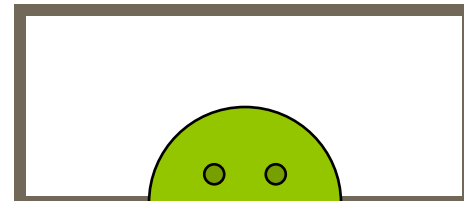
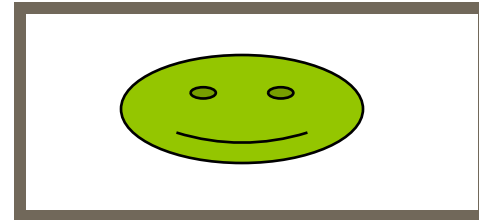
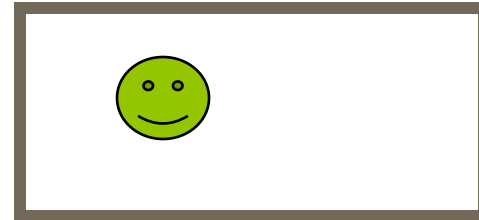
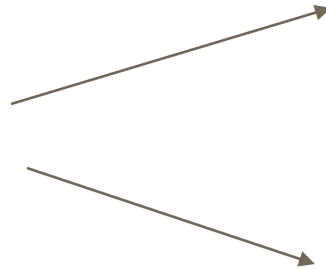
Reshaping the window

- We can reshape and resize the WebGL canvas by pulling the corner of the browser
- What happens to the display?
 - Must redraw from application
 - Two possibilities
 - Display part of world
 - Display whole world but force to fit in new window
 - Can alter aspect ratio

Reshape possibilities



original



reshaped

Reshaping the Canvas

```
resize(gl.canvas);  
gl.viewport(0, 0, gl.canvas.width,  
            gl.canvas.height);
```

```
function resize( canvas ) {  
    if(canvas.width !== canvas.clientWidth ||  
        canvas.height !== canvas.clientHeight) {  
        canvas.width = canvas.clientWidth;  
        canvas.height = canvas.clientHeight;  
    }  
}
```

- resize and gl.viewport need to be placed in the function that is argument of **requestedAnimationFrame** function